PRN232 – Final Assignment · FPT University · FA26

# EXE Team Matching System
# Thiết kế Database

## Bản 4 · PostgreSQL 16 · 1 instance · mỗi service 1 database riêng

Sinh tự động từ `deploy/db/*.sql` bằng `docs/db-design/build.py` ngày 02/11/2026.
Nguồn nghiệp vụ: `docs/ux/PROJECT_CONTEXT.md` (quy tắc 1–27, mục 5, 6, 9), sơ đồ C1 `docs/c4-architecture/c1.mmd`.

# 1. Tổng quan và quyền sở hữu

Theo C1, mỗi microservice sở hữu dữ liệu của mình và **không service nào đọc database của service khác**. Mỗi "DB" trong C1 là một database riêng trên cùng một instance PostgreSQL (container `postgres` trong Docker Compose), nên khi demo có thể mở từng database để xem. Mỗi service kết nối bằng một login role chỉ có quyền CONNECT vào database của mình; không có khóa ngoại chéo database. Script `deploy/db/00_init.sh` tạo role và database; các file `01–06_*.sql` tạo bảng.

Trong mỗi database, bảng nằm trong schema cùng tên service (ví dụ `team_db` → schema `team`), nên tài liệu ghi tên bảng dạng `schema.bảng`. Tổng cộng **73 bảng** nghiệp vụ.

| Service | "DB" trong C1 | Database | Schema | Login role | Bảng |
|---|---|---|---|---|---|
| Identity Service | Identity DB | `identity_db` | `identity` | `identity_svc` | 5: roles, user_accounts, user_account_roles, refresh_tokens, processed_events |
| Academic Service | Academic DB | `academic_db` | `academic` | `academic_svc` | 15: specializations, domains, class_codes, semesters, registration_rounds, students, student_skills, lecturers, lecturer_domains, mentors, mentor_domains, import_batches, semester_enrollments, rule_settings, outbox_messages |
| Team Service | Team DB | `team_db` | `team` | `team_svc` | 22: projects, project_versions, teams, team_members, team_baseline_members, team_domains, invitations, join_requests, join_request_ideas, recruitment_posts, recruitment_post_specializations, seeking_posts, seeking_post_domains, change_requests, change_request_items, topic_interests, lecturer_preferences, advisor_matching_runs, lecturer_assignments, mentor_assignments, audit_logs, outbox_messages |
| Notification Service | Notification DB | `notification_db` | `notification` | `notification_svc` | 4: contacts, notifications, email_logs, processed_events |

| Service | "DB" trong C1 | Database | Schema | Login role | Bảng |
|---|---|---|---|---|---|
| Collaboration Service | Collab DB | `collab_db` | `collab` | `collab_svc` | 27: workspaces, workspace_members, channels, messages, advisor_questions, message_reads, folders, project_files, file_versions, file_comments, message_attachments, meetings, meeting_slots, meeting_responses, meeting_minutes, meeting_reminders, meeting_schedules, schedule_windows, schedule_slots, slot_bookings, checkpoint_reviews, task_items, task_assignees, task_comments, deliverable_submissions, outbox_messages, processed_events |
| Job Service (Hangfire) | Job DB | `job_db` | `hangfire` | `job_svc` | Bảng do Hangfire.PostgreSql tự tạo (job, state, set, hash, lock…). Không có dữ liệu nghiệp vụ (mục 9). |
| Matching Service | — (chỉ Redis) | — | — | — | Không có SQL. Read model trong Redis (mục 9). |
| API Gateway | — | — | — | — | Không lưu dữ liệu. |

## 2. Quy ước chung

- **Đặt tên:** snake_case, bảng số nhiều; tiền tố `pk_` / `fk_` / `uq_` / `ck_` / `ex_` / `idx_` / `trg_`. Tên ràng buộc ổn định để service map lỗi PostgreSQL (`SqlState` 23505 / 23514 / 23503 + `ConstraintName`) sang ProblemDetails với mã lỗi trong `docs/contracts/README.md`.
- **Khóa chính:** `uuid` (`gen_random_uuid()`) cho dữ liệu nghiệp vụ vì ID đi qua Kafka và giữa các service; `integer IDENTITY` cho danh mục nhỏ; `bigint IDENTITY` cho audit log.
- **Thời gian:** luôn `timestamptz`, lưu UTC. Bảng có thay đổi có `created_at`, `updated_at` (trigger `trg_*_updated_at` tự cập nhật).
- **Trạng thái / enum:** `varchar` + `CHECK (... IN (...))`, không dùng kiểu ENUM của PostgreSQL.
- **Email:** `citext` để UNIQUE không phân biệt hoa thường. **SĐT:** `varchar(15)`, giữ số 0 đầu, chỉ có từ file import.
- **Tham chiếu chéo service:** chỉ lưu ID (không FK) + snapshot vài trường cần hiển thị / kiểm tra (MSSV, tên, chuyên ngành, mã domain). Cột như vậy có ghi chú `academic.students.id`… trong bảng cột.
- **Lịch sử:** dùng cột `status` / `ended_at` / `left_at` thay vì xóa cứng dữ liệu có ý nghĩa lịch sử.
- **Ràng buộc hai lớp:** Services (FluentValidation, Matching `ValidateComposition`) kiểm tra trước để trả lỗi thân thiện; CHECK / UNIQUE / trigger trong DB là lưới an toàn cuối, chống race condition khi nhiều request song song.
- **Optimistic concurrency:** cột hệ thống `xmin` làm row version cho `teams`, `invitations`, `join_requests`, `change_requests`, `slot_bookings`.
- **EF Core:** không dùng migration để tạo bảng. Entity và DbContext map đúng bảng trong script (`UseSnakeCaseNamingConvention`, `HasDefaultSchema`); muốn đổi bảng thì sửa script, chạy lại `build.py` và mở PR cho Thiên duyệt.

## 3. Quy tắc nghiệp vụ → cách ép trong DB

Số quy tắc theo PROJECT_CONTEXT §3. Quy tắc nào cần dữ liệu của service khác thì kiểm tra ở tầng Services.

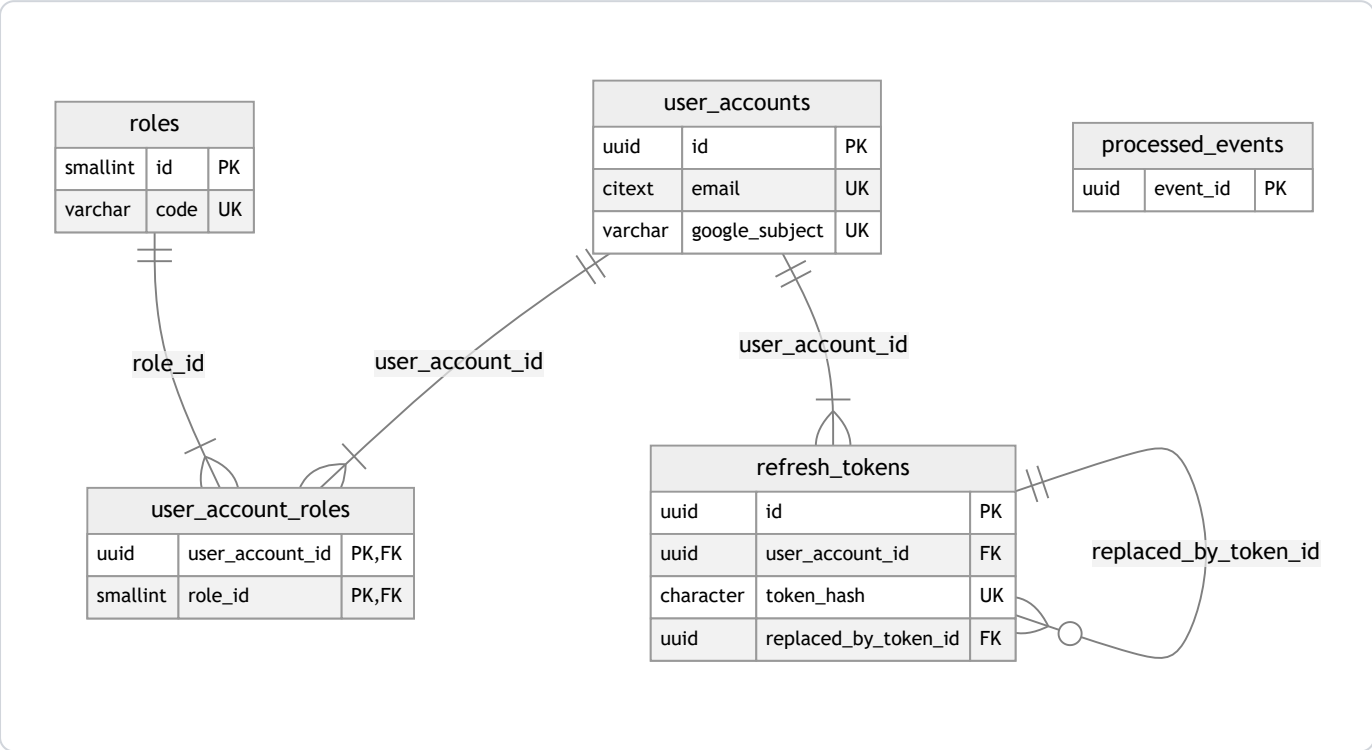
| Quy tắc | Cách thực thi |
|---|---|
| 1. Nhóm 4–6 người, đúng 1 leader là thành viên | `team_members.role` LEADER / MEMBER; partial unique `uq_team_members_one_leader` (≤ 1 leader); constraint trigger deferred `trg_team_members_composition` → `ck_team_exactly_one_leader` (= 1 leader lúc COMMIT, đổi leader trong một transaction được); `teams.member_count` do `trg_team_members_count` duy trì + `ck_teams_member_count` (0–6) chặn người thứ 7 ngay; khi FINALIZED trigger ép 4–6 (`ck_team_finalized_size`). |
| 2. ≥ 2 chuyên ngành khi chốt | Snapshot `team_members.specialization_id`; trigger `trg_teams_composition` → `ck_team_finalized_specializations` đếm DISTINCT khi status = FINALIZED (kể cả khi thay thành viên sau đó). |
| 3. Không cần cùng lớp | Không ràng buộc; lớp chỉ nằm ở `academic.semester_enrollments`. |
| 4. 1 sinh viên tối đa 1 nhóm mỗi học kỳ và môn | `uq_team_members_semester_student`; FK composite (`team_id, semester_id`) → `teams(id, semester_id)`. |
| 5. Chỉ tài khoản tạo sẵn từ danh sách được đăng nhập | `identity.user_accounts` (status PENDING → ACTIVE, `allowed_until`) + `user_account_roles`, tạo từ StudentImported / LecturerRegistered / MentorRegistered (idempotent qua `processed_events`). Login không gọi Academic. |
| 6. Chỉ thao tác khi đợt đăng ký mở | `academic.registration_rounds` (status, opens_at, deadline_at, `ex_registration_rounds_overlap` chống chồng đợt). Team hỏi Academic qua HTTP (Redis cache) trước khi ghi; không ép bằng DB vì nằm ở database khác. |
| 7. Đổi nhóm chỉ ở EXE201, có lý do, Admin duyệt | FK composite (`team_id, course_code`) + `ck_change_requests_course`; `ck_change_requests_reason`; `ck_change_requests_review`, `ck_change_requests_reject_note`; `uq_change_requests_pending` (1 yêu cầu chờ / nhóm). Thay đổi đã đồng ý nằm chờ trong `change_request_items` (`uq_change_request_items_pending`); mốc EXE101 ở `team_baseline_members`; `ck_change_requests_counts` lưu số và % thay đổi. |
| 8. Lời mời / yêu cầu hết hạn sau 72 giờ | `expires_at` + `ck_invitations_expiry`, `ck_join_requests_expiry`; partial index `idx_invitations_expiry`, `idx_join_requests_expiry` cho job quét; số giờ ở `rule_settings.invitation_expiry_hours`. |
| 9. Ghép giảng viên hướng dẫn tự động | `lecturer_preferences` (khóa chính team + rank 1–3, `uq_lecturer_preferences_lecturer`); `lecturer_assignments` với `reason`, `preference_rank`, `source` AUTO / MANUAL, `uq_lecturer_assignments_active` (1 giảng viên hiệu lực / nhóm); `advisor_matching_runs` lưu kế hoạch xem trước (Ghép sớm); số nhóm tối đa `lecturers.max_teams_exe101 / _exe201` (NULL = không giới hạn). |
| 10. Audit log mọi thay đổi nhóm | `team.audit_logs` ghi cùng transaction; `trg_audit_logs_append_only` chặn UPDATE / DELETE. |
| 11. MSSV 2 chữ + 6 số; SĐT dạng chuỗi | `ck_students_student_code`, `ck_team_members_student_code`; `ck_students_phone`. Quyền xem SĐT xử lý ở API. |
| 12. Chuyên ngành, domain từ danh mục | Trong academic: FK tới `specializations`, `domains` (`ck_domains_code` D01–D18). Trong team: lưu ID + mã có CHECK định dạng (`ck_team_domains_code`). |
| 13–15. Chốt nhóm, NeedsReview, trạng thái | `ck_teams_status` DRAFT / ACTIVE / NEEDS_REVIEW / FINALIZED / DISBANDED; `ck_teams_finalized_at`; `teams.is_auto_assigned`, `team_members.join_source` AUTO_ASSIGN / ADMIN cho ghép tự động và Admin sửa tay. |
| 16. Hồ sơ = 5 kỹ năng + mô tả | `academic.student_skills` (khóa chính student + skill, `ck_student_skills_skill`, `ck_student_skills_level` 1–5); `students.description` ≤ 500; `profile_completed_at`. |

| Quy tắc | Cách thực thi |
|---|---|
| 17. Ý tưởng khi xin vào nhóm | `join_requests.has_no_idea` + `join_request_ideas` (`ck_join_request_ideas_position` 1–3); constraint trigger `trg_join_requests_ideas` / `trg_join_request_ideas_check` ép "1–3 ý tưởng hoặc chưa có ý tưởng" lúc COMMIT. |
| 18. Khóa Khám phá sau khi lập nhóm | Giai đoạn tính từ `registration_rounds` (đợt TEAM_FORMATION của môn đã CLOSED). Team kiểm tra ở Services. |
| 19. Gợi ý đề tài, quan tâm của cố vấn | `team.topic_interests` (advisor_kind LECTURER / MENTOR, status OPEN / WITHDRAWN / CLOSED, `uq_topic_interests_open`); giới hạn ở `rule_settings.interest_extra_slots`, `max_interests_unlimited`; domain ở `lecturer_domains`, `mentor_domains`. |
| 20. Mentor | `academic.mentors` (`max_teams`, `preferred_teams` ≤ max, `is_internal` + `uq_mentors_lecturer_id`); `team.mentor_assignments` (`uq_mentor_assignments_active`: một mentor / nhóm); `semesters.mentoring_start_week`, `expected_mentor_sessions`; Identity: một tài khoản nhiều role (`user_account_roles`). |
| 21. Quyền workspace | `collab.workspace_members` dựng từ Kafka (role LEADER / MEMBER / INSTRUCTOR / MENTOR, `left_at`); `uq_workspace_members_one_instructor`, `uq_workspace_members_one_mentor`; `workspaces.previous_workspace_id` cho kho EXE101. |
| 22. Tin nhắn | `channels` (`uq_channels_workspace_kind`: TEAM, ADVISORS); `ck_messages_body` ≤ 2000; `advisor_questions` OPEN / ANSWERED; `message_reads`. |
| 23. Việc và bài nộp | `task_items` kind TASK / DELIVERABLE, `ck_task_items_deliverable_creator` (chỉ giảng viên tạo bài nộp), `deliverable_submissions` ACCEPTED / NEEDS_REVISION (`ck_deliverable_submissions_revision_note`). |
| 24. File | `project_files` (`ck_project_files_type`, `uq_project_files_name`), `file_versions` (`ck_file_versions_size` ≤ 25 MB); nội dung ở S3, DB chỉ giữ `object_key`. |
| 25. Họp | Lịch tuần EXE201: `meeting_schedules`, `schedule_windows`, `schedule_slots`, `slot_bookings` (`uq_slot_bookings_slot_active`: một nhóm / slot, ai đặt trước được trước; `uq_slot_bookings_workspace_active`); `meetings.week_no` + `uq_meetings_weekly`. Họp đề xuất: `meeting_slots`, `meeting_responses`. |
| 26. Đề tài xuyên kỳ | `team.projects`, `project_versions` (PIVOT phải có lý do và yêu cầu đổi nhóm: `ck_project_versions_pivot`); `teams.project_id` + `uq_teams_project_semester`; `change_request_items` action PROJECT_CHANGE (`uq_change_request_items_project_pending`). |
| 27. Checkpoint EXE101 (không chấm điểm) | `collab.checkpoint_reviews` (`ck_checkpoint_reviews_no` 1–4, `uq_checkpoint_reviews_workspace`); tuần mục tiêu ở `semesters.checkpoint_target_weeks`. |

# 4. Schema identity — Identity Service

Database `identity_db`, schema `identity`, login role `identity_svc` · 5 bảng · script `deploy/db/01_identity_db.sql`

Tài khoản được tạo sẵn từ sự kiện của Academic (quy tắc 5). Một tài khoản có thể giữ nhiều vai trò, ví dụ giảng viên vừa là mentor (quy tắc 20); `user_account_roles.external_ref_id` trỏ tới id sinh viên / giảng viên / mentor bên Academic và được phát vào claim `student_id / lecturer_id / mentor_id` của JWT. Refresh token chỉ lưu hash SHA-256, có rotation.




**`identity.roles`**

Vai trò: Student, Lecturer, Mentor, Admin. Leader là policy ở tầng Services, không phải role.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| `id` | `smallint` | | PK | | |
| `code` | `varchar(20)` | | UQ | | |
| `name` | `varchar(50)` | | | | |

**UNIQUE** `uq_roles_code` `UNIQUE (code)`

**PK** `pk_roles` `PRIMARY KEY (id)`

**CHECK** `ck_roles_code` `CHECK ((code IN ('Student', 'Lecturer', 'Admin', 'Mentor')))`

**`identity.user_accounts`**

Tài khoản đăng nhập kiêm email whitelist (rule 5). Student / Lecturer / Mentor được tạo sẵn (PENDING) từ sự kiện StudentImported / LecturerRegistered / MentorRegistered, chuyển ACTIVE ở lần đăng nhập Google đầu tiên. Vai trò nằm ở user_account_roles: một email có thể vừa là Lecturer vừa là Mentor (rule 20).

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| `id` | `uuid` | | PK | `gen_random_uuid()` | |
| `email` | `citext` | | UQ | | |
| `google_subject` | `varchar(255)` | ✓ | UQ | | |
| `full_name` | `varchar(150)` | | | | |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| status | varchar(20) | | | PENDING | |
| allowed_until | timestamptz | ✓ | | | hết hạn whitelist = end_date học kỳ |
| last_login_at | timestamptz | ✓ | | | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**UNIQUE** uq_user_accounts_email UNIQUE (email)

**UNIQUE** uq_user_accounts_google_subject UNIQUE (google_subject)

**PK** pk_user_accounts PRIMARY KEY (id)

**CHECK** ck_user_accounts_email CHECK (email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$')

**CHECK** ck_user_accounts_status CHECK ((status IN ('PENDING', 'ACTIVE', 'LOCKED')))

**TRIGGER** trg_user_accounts_updated_at BEFORE UPDATE ON identity.user_accounts FOR EACH ROW EXECUTE FUNCTION identity.set_updated_at()

#### identity.user_account_roles

Vai trò của tài khoản. external_ref_id = academic students.id / lecturers.id / mentors.id, đưa vào JWT (claim roles là danh sách; student_id / lecturer_id / mentor_id). Admin không có external_ref_id.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| user_account_id | uuid | | PK FK | | |
| role_id | smallint | | PK FK | | |
| external_ref_id | uuid | ✓ | | | |
| created_at | timestamptz | | | now() | |

**PK** pk_user_account_roles PRIMARY KEY (user_account_id, role_id)

**FK** fk_user_account_roles_account FOREIGN KEY (user_account_id) REFERENCES identity.user_accounts(id) ON DELETE CASCADE

**FK** fk_user_account_roles_role FOREIGN KEY (role_id) REFERENCES identity.roles(id) ON DELETE RESTRICT

**CHECK** ck_user_account_roles_external_ref CHECK (((role_id IN (1, 2, 4) = (external_ref_id IS NOT NULL)))

**INDEX** idx_user_account_roles_role_id btree (role_id)

**UNIQUE INDEX** uq_user_account_roles_external_ref btree (role_id, external_ref_id) WHERE (external_ref_id IS NOT NULL)

#### identity.refresh_tokens

Refresh token ~7 ngày, chỉ lưu SHA-256. Rotation: token cũ bị revoke và trỏ sang token mới.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| user_account_id | uuid | | FK | | |
| token_hash | character(64) | | UQ | | |
| expires_at | timestamptz | | | | |
| revoked_at | timestamptz | ✓ | | | |
| replaced_by_token_id | uuid | ✓ | FK | | |
| created_by_ip | inet | ✓ | | | |
| created_at | timestamptz | | | now() | |

**UNIQUE** uq_refresh_tokens_token_hash UNIQUE (token_hash)

**PK** pk_refresh_tokens PRIMARY KEY (id)

**FK** `fk_refresh_tokens_replaced_by` `FOREIGN KEY (replaced_by_token_id) REFERENCES identity.refresh_tokens(id) ON DELETE SET NULL`

**FK** `fk_refresh_tokens_user_account` `FOREIGN KEY (user_account_id) REFERENCES identity.user_accounts(id) ON DELETE CASCADE`

**CHECK** `ck_refresh_tokens_expiry` `CHECK (expires_at > created_at)`

**INDEX** `idx_refresh_tokens_replaced_by` `btree (replaced_by_token_id)`

**INDEX** `idx_refresh_tokens_user_account_id` `btree (user_account_id) WHERE (revoked_at IS NULL)`

#### `identity.processed_events`

Idempotency khi consume StudentImported / LecturerRegistered / MentorRegistered.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| event_id | uuid | | PK | | |
| event_type | varchar(100) | | | | |
| processed_at | timestamptz | | | now() | |

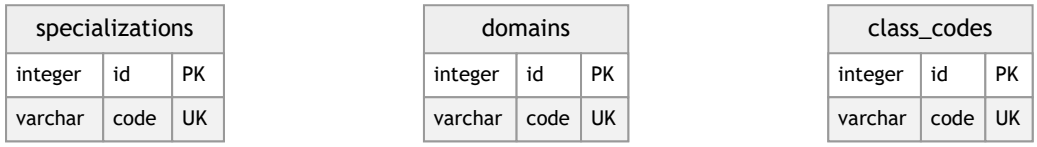
**PK** `pk_processed_events` `PRIMARY KEY (event_id)`

# 5. Schema academic — Academic Service

Database `academic_db`, schema `academic`, login role `academic_svc` · 15 bảng · script `deploy/db/02_academic_db.sql`

Nguồn của học kỳ, đợt đăng ký, danh sách sinh viên và giảng viên, mentor, danh mục. Publish sự kiện qua `outbox_messages` (topic `exe.student`, `exe.registration`).

## Danh mục (rule 12)




**academic.specializations**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | integer | | PK | IDENTITY | |
| code | varchar(20) | | UQ | | |
| name | varchar(100) | | | | |
| is_active | boolean | | | true | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**UNIQUE** `uq_specializations_code UNIQUE (code)`

**PK** `pk_specializations PRIMARY KEY (id)`

**CHECK** `ck_specializations_code CHECK (((code) ~ '^[A-Z0-9_]{2,20}$'))`

**TRIGGER** `trg_specializations_updated_at BEFORE UPDATE ON academic.specializations FOR EACH ROW EXECUTE FUNCTION academic.set_updated_at()`

**academic.domains**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | integer | | PK | IDENTITY | |
| code | varchar(3) | | UQ | | |
| name | varchar(150) | | | | |
| description | varchar(500) | ✓ | | | |
| is_active | boolean | | | true | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**UNIQUE** `uq_domains_code UNIQUE (code)`

**PK** `pk_domains PRIMARY KEY (id)`

**CHECK** `ck_domains_code CHECK (((code) ~ '^D(0[1-9]|1[0-8])$'))`

**TRIGGER** `trg_domains_updated_at BEFORE UPDATE ON academic.domains FOR EACH ROW EXECUTE FUNCTION academic.set_updated_at()`

**academic.class_codes**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | integer | | PK | IDENTITY | |
| code | varchar(20) | | UQ | | |
| is_active | boolean | | | true | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**UNIQUE** uq_class_codes_code UNIQUE (code)

**PK** pk_class_codes PRIMARY KEY (id)

**CHECK** ck_class_codes_code CHECK (((code) ~ '^[A-Z0-9]{3,20}$'))

**TRIGGER** trg_class_codes_updated_at BEFORE UPDATE ON academic.class_codes FOR EACH ROW EXECUTE FUNCTION academic.set_updated_at()

## Học kỳ, đợt đăng ký (rule 6)

**academic.semesters**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| code | varchar(4) | | UQ | | |
| name | varchar(50) | | | | |
| start_date | date | | | | |
| end_date | date | | | | |
| is_current | boolean | | | false | |
| mentoring_start_week | smallint | | | 6 | rule 20: mentor làm việc với nhóm từ tuần này |
| expected_mentor_sessions | smallint | | | 3 | số buổi mentor dự kiến ("Buổi mentor n/3") |
| slot_booking_deadline_week | smallint | | | 2 | rule 25: hạn chọn slot họp hằng tuần EXE201 (cuối tuần này) |
| meeting_slot_minutes | smallint | | | 15 | rule 25: độ dài một slot họp hằng tuần |
| checkpoint_target_weeks | smallint[] | ✓ | | | rule 27: tuần mục tiêu Checkpoint 1–4 (EXE101), NULL = không nhắc |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**UNIQUE** uq_semesters_code UNIQUE (code)

**PK** pk_semesters PRIMARY KEY (id)

**CHECK** ck_semesters_checkpoint_weeks CHECK (((checkpoint_target_weeks IS NULL) OR (cardinality(checkpoint_target_weeks) = 4)))

**CHECK** ck_semesters_code CHECK (((code) ~ '^(SP|SU|FA)[0-9]{2}$'))

**CHECK** ck_semesters_dates CHECK (end_date > start_date)

**CHECK** ck_semesters_mentor_sessions CHECK (((expected_mentor_sessions >= 1) AND (expected_mentor_sessions <= 10)))

**CHECK** ck_semesters_mentoring_week CHECK (((mentoring_start_week >= 1) AND (mentoring_start_week <= 20)))

**CHECK** ck_semesters_slot_booking_week CHECK (((slot_booking_deadline_week >= 1) AND (slot_booking_deadline_week <= 10)))

**CHECK** ck_semesters_slot_minutes CHECK (((meeting_slot_minutes >= 5) AND (meeting_slot_minutes <= 60)))

**UNIQUE INDEX** uq_semesters_is_current btree (is_current) WHERE is_current

**TRIGGER** trg_semesters_updated_at BEFORE UPDATE ON academic.semesters FOR EACH ROW EXECUTE FUNCTION academic.set_updated_at()

**academic.registration_rounds**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| semester_id | uuid | | FK | | |
| course_code | varchar(6) | | | | |
| round_type | varchar(20) | | | | |
| name | varchar(100) | | | | |
| opens_at | timestamptz | | | | |
| deadline_at | timestamptz | | | | |
| status | varchar(10) | | | SCHEDULED | |
| closed_at | timestamptz | ✓ | | | |
| created_by | uuid | | | | identity.user_accounts.id (claim sub) |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**EXCLUDE** ex_registration_rounds_overlap EXCLUDE USING gist (semester_id WITH =, course_code WITH =, round_type WITH =, tstzrange(opens_at, deadline_at, '[)') WITH &&)

**PK** pk_registration_rounds PRIMARY KEY (id)

**FK** fk_registration_rounds_semester FOREIGN KEY (semester_id) REFERENCES academic.semesters(id) ON DELETE RESTRICT

**CHECK** ck_registration_rounds_closed CHECK ((((status) = 'CLOSED') = (closed_at IS NOT NULL)))

**CHECK** ck_registration_rounds_course CHECK ((course_code IN ('EXE101', 'EXE201')))

**CHECK** ck_registration_rounds_course_type CHECK ((((round_type) <> 'CHANGE_REQUEST') OR ((course_code) = 'EXE201')))

**CHECK** ck_registration_rounds_status CHECK ((status IN ('SCHEDULED', 'OPEN', 'CLOSED')))

**CHECK** ck_registration_rounds_type CHECK ((round_type IN ('TEAM_FORMATION', 'CHANGE_REQUEST')))

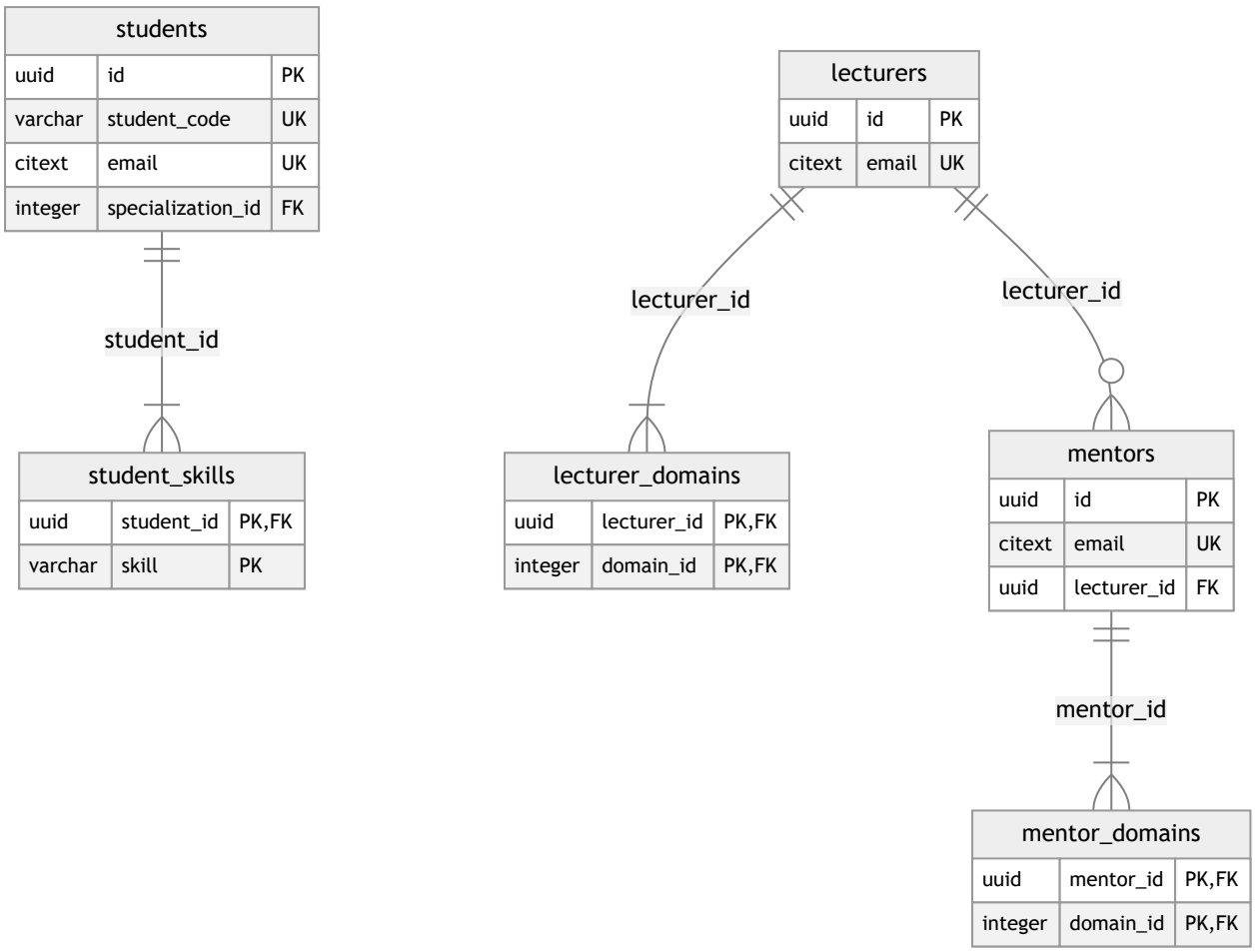
**CHECK** ck_registration_rounds_window CHECK (deadline_at > opens_at)

**INDEX** idx_registration_rounds_open btree (deadline_at) WHERE ((status) = 'OPEN')

**INDEX** idx_registration_rounds_semester_id btree (semester_id)

**TRIGGER** trg_registration_rounds_updated_at BEFORE UPDATE ON academic.registration_rounds FOR EACH ROW EXECUTE FUNCTION academic.set_updated_at()

## Sinh viên, giảng viên




**`academic.students`**

Hồ sơ sinh viên, ổn định qua các học kỳ (EXE101 -> EXE201). MSSV 2 chữ + 6 số. Rule 11, 16: SĐT chỉ đến từ file import (chuỗi, không cho sinh viên sửa); hồ sơ do sinh viên sửa = 5 kỹ năng + mô tả.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| `id` | `uuid` | | PK | `gen_random_uuid()` | |
| `student_code` | `varchar(8)` | | UQ | | |
| `full_name` | `varchar(150)` | | | | |
| `email` | `citext` | | UQ | | |
| `phone` | `varchar(15)` | ✓ | | | từ file import |
| `specialization_id` | `integer` | | FK | | |
| `description` | `varchar(500)` | ✓ | | | giới thiệu ngắn (rule 16) |
| `profile_completed_at` | `timestamptz` | ✓ | | | hoàn tất A05 (đã đánh giá đủ 5 kỹ năng) |
| `is_seeking_team` | `boolean` | | | `false` | công tắc "Tôi đang tìm nhóm" (S03) |
| `created_at` | `timestamptz` | | | `now()` | |
| `updated_at` | `timestamptz` | | | `now()` | |

**UNIQUE** `uq_students_email` `UNIQUE (email)`

**UNIQUE** `uq_students_student_code` `UNIQUE (student_code)`

**PK** `pk_students` `PRIMARY KEY (id)`

**FK** fk_students_specialization FOREIGN KEY (specialization_id) REFERENCES academic.specializations(id) ON DELETE RESTRICT

**CHECK** ck_students_email CHECK (email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$')

**CHECK** ck_students_phone CHECK (((phone IS NULL) OR ((phone) ~ '^\+?[0-9]{9,14}$')))

**CHECK** ck_students_student_code CHECK (((student_code) ~ '^[A-Z]{2}[0-9]{6}$'))

**INDEX** idx_students_full_name btree (lower(full_name))

**INDEX** idx_students_specialization_id btree (specialization_id)

**TRIGGER** trg_students_updated_at BEFORE UPDATE ON academic.students FOR EACH ROW EXECUTE FUNCTION academic.set_updated_at()

#### academic.student_skills

5 kỹ năng cố định, tự đánh giá 1–5 (rule 16). Thiếu dòng = chưa đánh giá.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| student_id | uuid | | PK FK | | |
| skill | varchar(25) | | PK | | |
| level | smallint | | | | |
| updated_at | timestamptz | | | now() | |

**PK** pk_student_skills PRIMARY KEY (student_id, skill)

**FK** fk_student_skills_student FOREIGN KEY (student_id) REFERENCES academic.students(id) ON DELETE CASCADE

**CHECK** ck_student_skills_level CHECK (((level >= 1) AND (level <= 5)))

**CHECK** ck_student_skills_skill CHECK ((skill IN ('TECHNOLOGY', 'UI_UX_DESIGN', 'MANAGEMENT', 'MARKETING', 'COMMUNICATION_PLANNING')))

**TRIGGER** trg_student_skills_updated_at BEFORE UPDATE ON academic.student_skills FOR EACH ROW EXECUTE FUNCTION academic.set_updated_at()

#### academic.lecturers

Giảng viên do Admin nhập email (cấp role Lecturer qua sự kiện LecturerRegistered).

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| full_name | varchar(150) | | | | |
| email | citext | | UQ | | |
| phone | varchar(15) | ✓ | | | |
| max_teams_exe101 | smallint | ✓ | | | rule 9: số nhóm tối đa do Admin đặt, NULL = không giới hạn |
| max_teams_exe201 | smallint | ✓ | | | |
| is_active | boolean | | | true | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**UNIQUE** uq_lecturers_email UNIQUE (email)

**PK** pk_lecturers PRIMARY KEY (id)

**CHECK** ck_lecturers_email CHECK (email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$')

**CHECK** ck_lecturers_max_teams CHECK ((((max_teams_exe101 IS NULL) OR ((max_teams_exe101 >= 0) AND (max_teams_exe101 <= 200))) AND ((max_teams_exe201 IS NULL) OR ((max_teams_exe201 >= 0) AND (max_teams_exe201 <= 200)))))

**CHECK** ck_lecturers_phone CHECK (((phone IS NULL) OR ((phone) ~ '^\+?[0-9]{9,14}$')))

**TRIGGER** trg_lecturers_updated_at BEFORE UPDATE ON academic.lecturers FOR EACH ROW EXECUTE FUNCTION academic.set_updated_at()

**academic.lecturer_domains**

Chuyên môn của giảng viên chọn từ danh mục domain (rule 12) – đầu vào gợi ý đề tài (rule 19, L07).

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| lecturer_id | uuid | | PK FK | | |
| domain_id | integer | | PK FK | | |

**PK** pk_lecturer_domains PRIMARY KEY (lecturer_id, domain_id)

**FK** fk_lecturer_domains_domain FOREIGN KEY (domain_id) REFERENCES academic.domains(id) ON DELETE RESTRICT

**FK** fk_lecturer_domains_lecturer FOREIGN KEY (lecturer_id) REFERENCES academic.lecturers(id) ON DELETE CASCADE

**INDEX** idx_lecturer_domains_domain btree (domain_id)

**academic.mentors**

Mentor do Bộ môn tạo (rule 20), phát MentorRegistered. Người ngoài trường hoặc giảng viên của trường (lecturer_id: cùng người với một giảng viên – một tài khoản, hai vai trò). max_teams do Bộ môn đặt riêng từng mentor; preferred_teams do mentor tự chọn (≤ max_teams).

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| full_name | varchar(150) | | | | |
| email | citext | | UQ | | |
| organization | varchar(150) | ✓ | | | |
| is_internal | boolean | | | false | |
| lecturer_id | uuid | ✓ | FK | | |
| description | varchar(500) | ✓ | | | |
| max_teams | smallint | | | | |
| preferred_teams | smallint | ✓ | | | |
| is_active | boolean | | | true | |
| created_by | uuid | | | | identity.user_accounts.id (Admin) |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**UNIQUE** uq_mentors_email UNIQUE (email)

**PK** pk_mentors PRIMARY KEY (id)

**FK** fk_mentors_lecturer FOREIGN KEY (lecturer_id) REFERENCES academic.lecturers(id) ON DELETE SET NULL

**CHECK** ck_mentors_email CHECK (email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$')

**CHECK** ck_mentors_internal CHECK (((lecturer_id IS NULL) OR is_internal))

**CHECK** ck_mentors_max_teams CHECK (((max_teams >= 1) AND (max_teams <= 50)))

**CHECK** ck_mentors_preferred_teams CHECK (((preferred_teams IS NULL) OR ((preferred_teams >= 1) AND (preferred_teams <= max_teams))))

**UNIQUE INDEX** uq_mentors_lecturer_id btree (lecturer_id) WHERE (lecturer_id IS NOT NULL)

**TRIGGER** trg_mentors_updated_at BEFORE UPDATE ON academic.mentors FOR EACH ROW EXECUTE FUNCTION academic.set_updated_at()

**academic.mentor_domains**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| mentor_id | uuid | | PK FK | | |
| domain_id | integer | | PK FK | | |

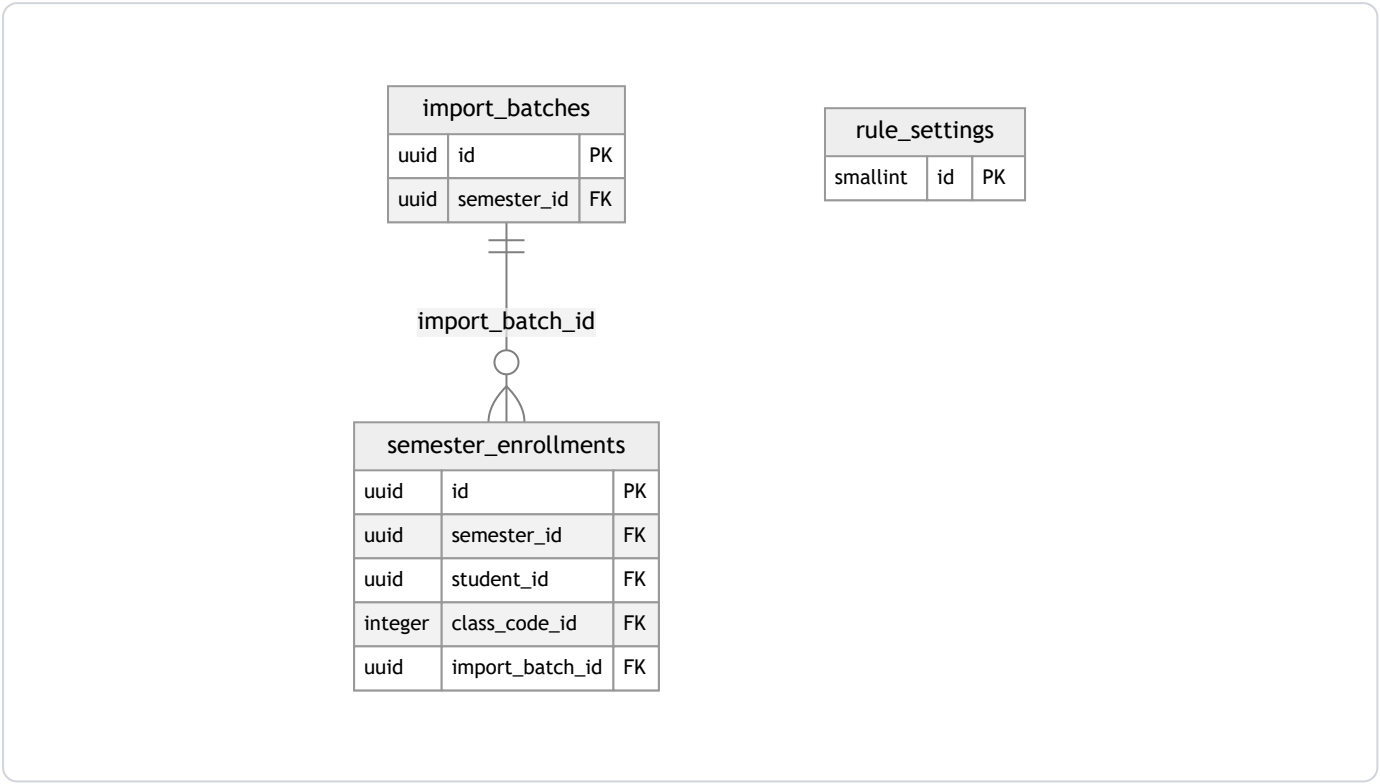
**PK** pk_mentor_domains PRIMARY KEY (mentor_id, domain_id)

**FK** fk_mentor_domains_domain FOREIGN KEY (domain_id) REFERENCES academic.domains(id) ON DELETE RESTRICT

**FK** fk_mentor_domains_mentor FOREIGN KEY (mentor_id) REFERENCES academic.mentors(id) ON DELETE CASCADE

**INDEX** idx_mentor_domains_domain btree (domain_id)

## Import (W06–W08) và danh sách sinh viên học kỳ (nguồn whitelist)




**academic.import_batches**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| semester_id | uuid | | FK | | |
| file_name | varchar(255) | | | | |
| total_rows | integer | | | 0 | |
| success_rows | integer | | | 0 | |
| failed_rows | integer | | | 0 | |
| errors | jsonb | ✓ | | | [{row, field, message}] |
| status | varchar(20) | | | PROCESSING | |
| imported_by | uuid | | | | identity.user_accounts.id |
| imported_by_name | varchar(150) | ✓ | | | snapshot tên người import (W08); tên nằm ở Identity |
| created_at | timestamptz | | | now() | |
| completed_at | timestamptz | ✓ | | | |

**PK** pk_import_batches PRIMARY KEY (id)

**FK** fk_import_batches_semester FOREIGN KEY (semester_id) REFERENCES academic.semesters(id) ON DELETE RESTRICT

**CHECK** ck_import_batches_rows CHECK (((total_rows >= 0) AND (success_rows >= 0) AND (failed_rows >= 0) AND ((success_rows + failed_rows) <= total_rows)))

**CHECK** ck_import_batches_status CHECK ((status IN ('PROCESSING', 'COMPLETED', 'FAILED')))

**INDEX** idx_import_batches_semester_id btree (semester_id)

**academic.semester_enrollments**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| semester_id | uuid | | UQ FK | | |
| student_id | uuid | | UQ FK | | |
| course_code | varchar(6) | | | | |
| class_code_id | integer | | FK | | |
| import_batch_id | uuid | ✓ | FK | | |
| created_at | timestamptz | | | now() | |

**UNIQUE** uq_semester_enrollments_semester_student UNIQUE (semester_id, student_id)

**PK** pk_semester_enrollments PRIMARY KEY (id)

**FK** fk_semester_enrollments_class_code FOREIGN KEY (class_code_id) REFERENCES academic.class_codes(id) ON DELETE RESTRICT

**FK** fk_semester_enrollments_import_batch FOREIGN KEY (import_batch_id) REFERENCES academic.import_batches(id) ON DELETE SET NULL

**FK** fk_semester_enrollments_semester FOREIGN KEY (semester_id) REFERENCES academic.semesters(id) ON DELETE RESTRICT

**FK** fk_semester_enrollments_student FOREIGN KEY (student_id) REFERENCES academic.students(id) ON DELETE RESTRICT

**CHECK** ck_semester_enrollments_course CHECK ((course_code IN ('EXE101', 'EXE201')))

**INDEX** idx_semester_enrollments_class_code_id btree (class_code_id)

**INDEX** idx_semester_enrollments_import_batch btree (import_batch_id)

**INDEX** idx_semester_enrollments_student_id btree (student_id)

**academic.rule_settings**

Cấu hình quy tắc (W05). Một dòng duy nhất (id = 1).

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | smallint | | PK | 1 | |
| min_team_size | smallint | | | 4 | |
| max_team_size | smallint | | | 6 | |
| min_specializations | smallint | | | 2 | |
| invitation_expiry_hours | smallint | | | 72 | |
| reminder_hours | smallint[] | | | {72,24} | |
| max_ideas_per_join_request | smallint | | | 3 | rule 17 |
| interest_extra_slots | smallint | | | 2 | rule 19: giới hạn quan tâm = số nhóm còn nhận + n (mentor: tối đa + n) |
| max_interests_unlimited | smallint | | | 10 | rule 19: giới hạn quan tâm khi giảng viên không có số nhóm tối đa |
| max_lecturer_preferences | smallint | | | 3 | rule 9: số nguyện vọng giảng viên của nhóm |
| max_file_size_mb | smallint | | | 25 | rule 24 |
| updated_by | uuid | ✓ | | | |
| updated_at | timestamptz | | | now() | |

**PK** pk_rule_settings PRIMARY KEY (id)

**CHECK** ck_rule_settings_expiry CHECK (invitation_expiry_hours > 0)

**CHECK** ck_rule_settings_file_size CHECK (((max_file_size_mb >= 1) AND (max_file_size_mb <= 100)))

**CHECK** ck_rule_settings_ideas CHECK (((max_ideas_per_join_request >= 1) AND (max_ideas_per_join_request <= 10)))

**CHECK** ck_rule_settings_interest CHECK ((((interest_extra_slots >= 0) AND (interest_extra_slots <= 10)) AND ((max_interests_unlimited >= 1) AND (max_interests_unlimited <= 100))))

**CHECK** ck_rule_settings_preferences CHECK (((max_lecturer_preferences >= 1) AND (max_lecturer_preferences <= 10)))

**CHECK** ck_rule_settings_singleton CHECK (id = 1)

**CHECK** ck_rule_settings_team_size CHECK (((min_team_size >= 2) AND (max_team_size >= min_team_size)))

**TRIGGER** trg_rule_settings_updated_at BEFORE UPDATE ON academic.rule_settings FOR EACH ROW EXECUTE FUNCTION academic.set_updated_at()

## Transactional Outbox

**academic.outbox_messages**

StudentImported, StudentProfileUpdated, LecturerRegistered, LecturerProfileUpdated, MentorRegistered (topic exe.student); RoundOpened, RoundClosed (exe.registration).

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | | = eventId của envelope |
| topic | varchar(100) | | | | |
| event_type | varchar(100) | | | | |
| aggregate_type | varchar(50) | | | | |
| aggregate_id | uuid | | | | Kafka message key |
| payload | jsonb | | | | |
| correlation_id | varchar(64) | ✓ | | | |
| occurred_at | timestamptz | | | now() | |
| processed_at | timestamptz | ✓ | | | |
| attempts | integer | | | 0 | |
| last_error | text | ✓ | | | |

**PK** pk_outbox_messages PRIMARY KEY (id)

**CHECK** ck_outbox_messages_attempts CHECK (attempts >= 0)

**INDEX** idx_outbox_messages_pending btree (occurred_at) WHERE (processed_at IS NULL)

# 6. Schema team — Team Service

Database `team_db`, schema `team`, login role `team_svc` · 22 bảng · script `deploy/db/03_team_db.sql`

Nghiệp vụ lõi: đề tài, nhóm, lời mời, yêu cầu tham gia, bài đăng, đổi nhóm, ghép cố vấn. Các trigger ép quy tắc 1, 2, 17 và audit log append-only. Publish qua `outbox_messages` (topic `exe.team`, `exe.invitation`, `exe.change-request`, `exe.assignment`).

## Đề tài xuyên kỳ (rule 26)

**`team.projects`**

Đề tài tạo khi nhóm EXE101 đăng ký ý tưởng; nhóm EXE201 kế thừa trỏ cùng project_id.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| name | varchar(200) | | | | |
| description | varchar(4000) | ✓ | | | |
| origin_semester_id | uuid | | | | academic.semesters.id |
| origin_course_code | varchar(6) | | | | |
| current_version | smallint | | | 1 | |
| status | varchar(10) | | | ACTIVE | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**PK** pk_projects PRIMARY KEY (id)

**CHECK** ck_projects_course CHECK ((origin_course_code IN ('EXE101', 'EXE201')))

**CHECK** ck_projects_name CHECK ((char_length(btrim(name)) >= 2))

**CHECK** ck_projects_status CHECK ((status IN ('ACTIVE', 'CLOSED')))

**CHECK** ck_projects_version CHECK (current_version >= 1)

**INDEX** idx_projects_name btree (lower(name))

**TRIGGER** trg_projects_updated_at BEFORE UPDATE ON team.projects FOR EACH ROW EXECUTE FUNCTION team.set_updated_at()

**`team.project_versions`**

Lịch sử đề tài. PIVOT (đổi đề tài ở EXE201) đi qua yêu cầu đổi nhóm và cần lý do.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| project_id | uuid | | PK FK | | |
| version | smallint | | PK | | |
| name | varchar(200) | | | | |
| description | varchar(4000) | ✓ | | | |
| change_type | varchar(10) | | | | |
| reason | varchar(1000) | ✓ | | | |
| semester_id | uuid | | | | |
| course_code | varchar(6) | | | | |
| change_request_id | uuid | ✓ | | | |
| changed_by_account_id | uuid | ✓ | | | NULL = hệ thống (import) |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| created_at | timestamptz | | | now() | |

**PK** pk_project_versions PRIMARY KEY (project_id, version)

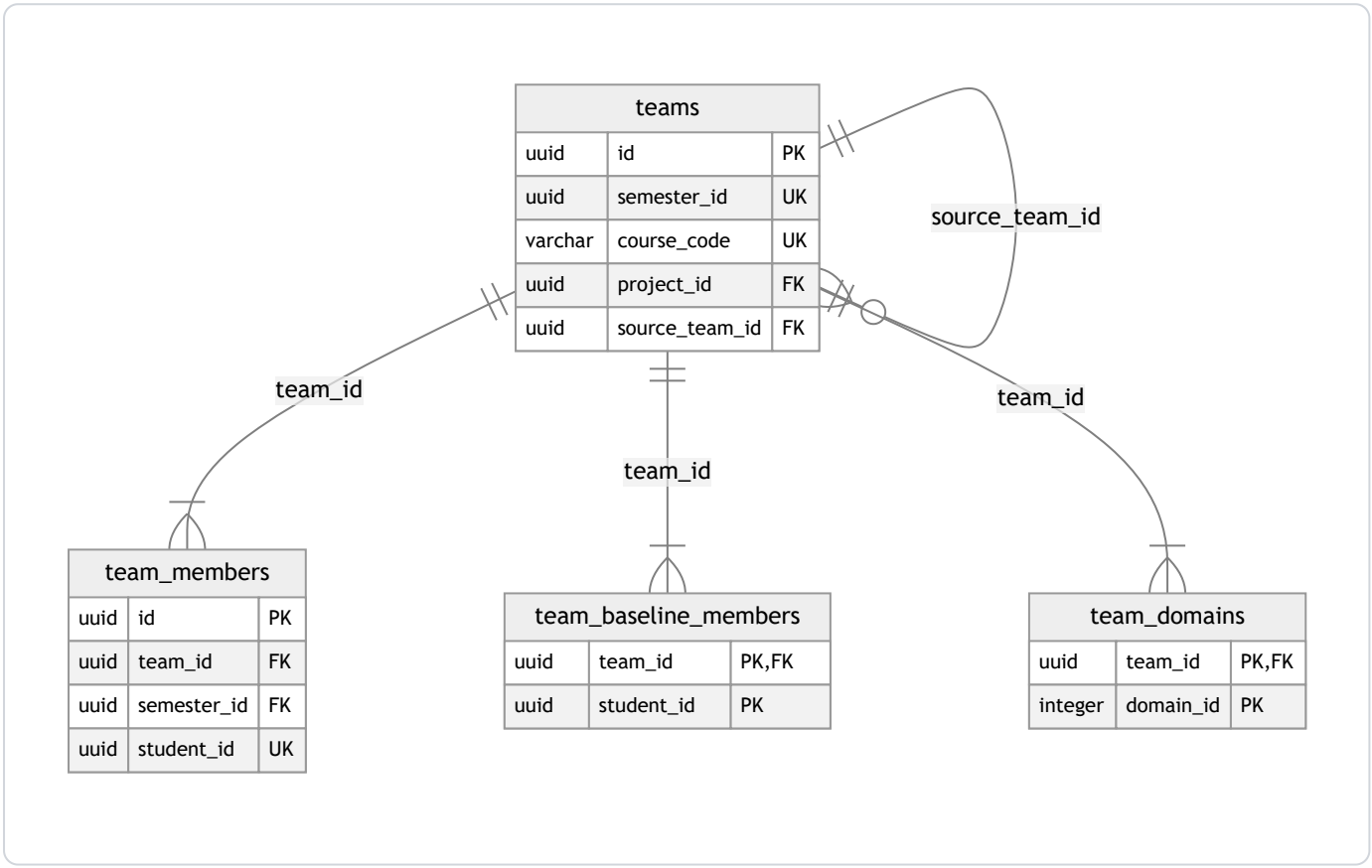
**FK** fk_project_versions_project FOREIGN KEY (project_id) REFERENCES team.projects(id) ON DELETE CASCADE

**CHECK** ck_project_versions_course CHECK ((course_code IN ('EXE101', 'EXE201')))

**CHECK** ck_project_versions_pivot CHECK ((((change_type) <> 'PIVOT') OR ((reason IS NOT NULL) AND (change_request_id IS NOT NULL))))

**CHECK** ck_project_versions_type CHECK ((change_type IN ('CREATED', 'EDITED', 'PIVOT')))

## Nhóm và thành viên (rule 1, 2, 4, 13, 15)




**team.teams**

status: DRAFT (< 4 người) -> ACTIVE -> FINALIZED, hoặc NEEDS_REVIEW sau hạn chót; DISBANDED khi giải tán.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | UQ PK | gen_random_uuid() | |
| semester_id | uuid | | UQ | | academic.semesters.id |
| course_code | varchar(6) | | UQ | | |
| name | varchar(100) | | | | |
| project_id | uuid | ✓ | FK | | đề tài (rule 26); NULL khi nhóm chưa đăng ký ý tưởng |
| status | varchar(12) | | | DRAFT | |
| member_count | smallint | | | 0 | trigger duy trì |
| is_inherited | boolean | | | false | nhóm EXE201 kế thừa từ mốc EXE101 (import) |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| source_team_id | uuid | ✓ | FK | | nhóm EXE101 gốc (cùng đề tài) nếu có trong hệ thống |
| is_auto_assigned | boolean | | | false | |
| finalized_at | timestamptz | ✓ | | | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**UNIQUE** uq_teams_id_course UNIQUE (id, course_code)

**UNIQUE** uq_teams_id_semester UNIQUE (id, semester_id)

**TRIGGER** trg_teams_composition TRIGGER DEFERRABLE INITIALLY DEFERRED

**PK** pk_teams PRIMARY KEY (id)

**FK** fk_teams_project FOREIGN KEY (project_id) REFERENCES team.projects(id) ON DELETE RESTRICT

**FK** fk_teams_source_team FOREIGN KEY (source_team_id) REFERENCES team.teams(id) ON DELETE RESTRICT

**CHECK** ck_teams_course CHECK ((course_code IN ('EXE101', 'EXE201')))

**CHECK** ck_teams_finalized_at CHECK ((((status) <> 'FINALIZED') OR (finalized_at IS NOT NULL)))

**CHECK** ck_teams_inherited CHECK ((((NOT is_inherited) AND (source_team_id IS NULL)) OR ((course_code) = 'EXE201')))

**CHECK** ck_teams_member_count CHECK (((member_count >= 0) AND (member_count <= 6)))

**CHECK** ck_teams_name CHECK ((char_length(btrim(name)) >= 2))

**CHECK** ck_teams_status CHECK ((status IN ('DRAFT', 'ACTIVE', 'NEEDS_REVIEW', 'FINALIZED', 'DISBANDED')))

**INDEX** idx_teams_semester_member_count btree (semester_id, member_count) WHERE (status IN ('DRAFT', 'ACTIVE'))

**INDEX** idx_teams_semester_status_name btree (semester_id, status, name)

**UNIQUE INDEX** uq_teams_project_semester btree (project_id, semester_id) WHERE ((project_id IS NOT NULL) AND ((status) <> 'DISBANDED'))

**UNIQUE INDEX** uq_teams_semester_name btree (semester_id, course_code, lower(name)) WHERE ((status) <> 'DISBANDED')

**UNIQUE INDEX** uq_teams_source_team_id btree (source_team_id) WHERE (source_team_id IS NOT NULL)

**TRIGGER** trg_teams_composition AFTER INSERT OR UPDATE OF status ON team.teams DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION team.tg_teams_composition()

**TRIGGER** trg_teams_updated_at BEFORE UPDATE ON team.teams FOR EACH ROW EXECUTE FUNCTION team.set_updated_at()

#### team.team_members

Thành viên hiện tại. UNIQUE (semester_id, student_id) = rule 4. Snapshot MSSV, tên, chuyên ngành.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| team_id | uuid | | FK | | |
| semester_id | uuid | | UQ FK | | |
| student_id | uuid | | UQ | | academic.students.id |
| student_code | varchar(8) | | | | |
| full_name | varchar(150) | | | | |
| specialization_id | integer | | | | |
| role | varchar(10) | | | MEMBER | |
| join_source | varchar(20) | | | | |
| joined_at | timestamptz | | | now() | |

**UNIQUE** uq_team_members_semester_student UNIQUE (semester_id, student_id)

**TRIGGER** trg_team_members_composition TRIGGER DEFERRABLE INITIALLY DEFERRED

**PK** pk_team_members PRIMARY KEY (id)

**FK** fk_team_members_team FOREIGN KEY (team_id, semester_id) REFERENCES team.teams(id, semester_id) ON DELETE CASCADE

**CHECK** ck_team_members_join_source CHECK ((join_source IN ('CREATOR', 'INVITATION', 'JOIN_REQUEST', 'CHANGE_REQUEST', 'AUTO_ASSIGN', 'INHERITED', 'ADMIN')))

**CHECK** ck_team_members_role CHECK ((role IN ('LEADER', 'MEMBER')))

**CHECK** ck_team_members_student_code CHECK (((student_code) ~ '^[A-Z]{2}[0-9]{6}$'))

**INDEX** idx_team_members_team_id btree (team_id)

**UNIQUE INDEX** uq_team_members_one_leader btree (team_id) WHERE ((role) = 'LEADER')

**TRIGGER** trg_team_members_composition AFTER INSERT OR DELETE OR UPDATE ON team.team_members DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION team.tg_team_members_composition()

**TRIGGER** trg_team_members_count AFTER INSERT OR DELETE ON team.team_members FOR EACH ROW EXECUTE FUNCTION team.tg_team_members_count()

**TRIGGER** trg_team_members_immutable BEFORE UPDATE ON team.team_members FOR EACH ROW EXECUTE FUNCTION team.tg_team_members_immutable()

#### team.team_baseline_members

Mốc EXE101 của nhóm EXE201 kế thừa (import Excel, BE baseline-import). Tham chiếu cho yêu cầu đổi nhóm.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| team_id | uuid | | PK FK | | |
| student_id | uuid | | PK | | |
| student_code | varchar(8) | | | | |
| full_name | varchar(150) | | | | |
| specialization_id | integer | | | | |
| role | varchar(10) | | | | |
| imported_at | timestamptz | | | now() | |

**PK** pk_team_baseline_members PRIMARY KEY (team_id, student_id)

**FK** fk_team_baseline_members_team FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE CASCADE

**CHECK** ck_team_baseline_members_code CHECK (((student_code) ~ '^[A-Z]{2}[0-9]{6}$'))

**CHECK** ck_team_baseline_members_role CHECK ((role IN ('LEADER', 'MEMBER')))

**UNIQUE INDEX** uq_team_baseline_members_leader btree (team_id) WHERE ((role) = 'LEADER')

#### team.team_domains

Domain của nhóm (từ danh mục). domain_code snapshot để lọc ?domain=D04.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| team_id | uuid | | PK FK | | |
| domain_id | integer | | PK | | academic.domains.id |
| domain_code | varchar(3) | | | | |
| is_primary | boolean | | | false | |

**PK** pk_team_domains PRIMARY KEY (team_id, domain_id)

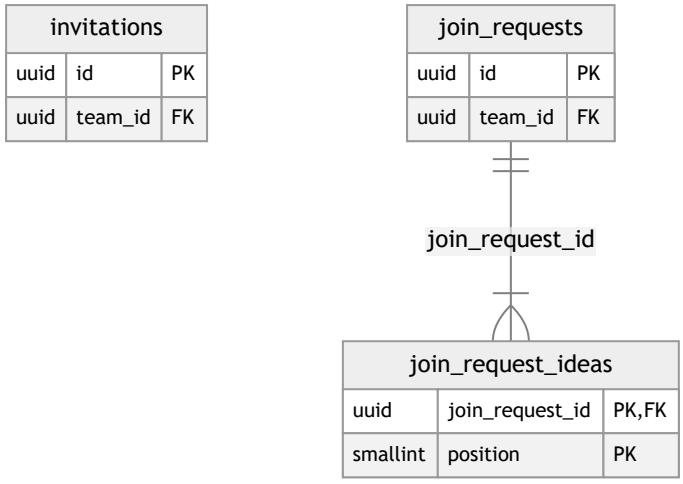
**FK** fk_team_domains_team FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE CASCADE

**CHECK** ck_team_domains_code CHECK (((domain_code) ~ '^D(0[1-9]|1[0-8])$'))

**INDEX** idx_team_domains_domain_code btree (domain_code)

**UNIQUE INDEX** uq_team_domains_primary btree (team_id) WHERE is_primary

## Lời mời, yêu cầu tham gia (rule 8: hết hạn 72 giờ)




**team.invitations**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| team_id | uuid | | FK | | |
| invitee_student_id | uuid | | | | |
| invited_by_student_id | uuid | | | | leader |
| message | varchar(500) | ✓ | | | |
| status | varchar(10) | | | PENDING | |
| created_at | timestamptz | | | now() | |
| expires_at | timestamptz | | | (now() + 72:00:00) | |
| resolved_at | timestamptz | ✓ | | | |

**PK** pk_invitations PRIMARY KEY (id)

**FK** fk_invitations_team FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE CASCADE

**CHECK** ck_invitations_expiry CHECK (expires_at > created_at)

**CHECK** ck_invitations_resolved CHECK ((((status) = 'PENDING') = (resolved_at IS NULL)))

**CHECK** ck_invitations_self CHECK (invitee_student_id <> invited_by_student_id)

**CHECK** ck_invitations_status CHECK ((status IN ('PENDING', 'ACCEPTED', 'DECLINED', 'EXPIRED', 'CANCELLED')))

**INDEX** idx_invitations_expiry btree (expires_at) WHERE ((status) = 'PENDING')

**INDEX** idx_invitations_invitee btree (invitee_student_id, status)

**UNIQUE INDEX** uq_invitations_pending btree (team_id, invitee_student_id) WHERE ((status) = 'PENDING')

**team.join_requests**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| team_id | uuid | | FK | | |
| student_id | uuid | | | | |
| message | varchar(500) | ✓ | | | |
| has_no_idea | boolean | | | false | tích "Chưa có ý tưởng" (rule 17) |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| status | varchar(10) | | | PENDING | |
| resolved_by_student_id | uuid | ✓ | | | leader duyệt |
| created_at | timestamptz | | | now() | |
| expires_at | timestamptz | | | (now() + 72:00:00) | |
| resolved_at | timestamptz | ✓ | | | |

**TRIGGER** trg_join_requests_ideas TRIGGER DEFERRABLE INITIALLY DEFERRED

**PK** pk_join_requests PRIMARY KEY (id)

**FK** fk_join_requests_team FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE CASCADE

**CHECK** ck_join_requests_expiry CHECK (expires_at > created_at)

**CHECK** ck_join_requests_resolved CHECK ((((status) = 'PENDING') = (resolved_at IS NULL)))

**CHECK** ck_join_requests_status CHECK ((status IN ('PENDING', 'ACCEPTED', 'DECLINED', 'EXPIRED', 'CANCELLED')))

**INDEX** idx_join_requests_expiry btree (expires_at) WHERE ((status) = 'PENDING')

**INDEX** idx_join_requests_student btree (student_id, status)

**UNIQUE INDEX** uq_join_requests_pending btree (team_id, student_id) WHERE ((status) = 'PENDING')

**TRIGGER** trg_join_requests_ideas AFTER INSERT OR UPDATE OF has_no_idea ON team.join_requests DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION team.tg_join_requests_ideas()

#### team.join_request_ideas

Ý tưởng kèm yêu cầu tham gia (rule 17): 1–3 ý tưởng hoặc has_no_idea = true, kiểm tra lúc COMMIT.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| join_request_id | uuid | | PK FK | | |
| position | smallint | | PK | | |
| title | varchar(80) | | | | |
| description | varchar(300) | ✓ | | | |

**TRIGGER** trg_join_request_ideas_check TRIGGER DEFERRABLE INITIALLY DEFERRED

**PK** pk_join_request_ideas PRIMARY KEY (join_request_id, "position")

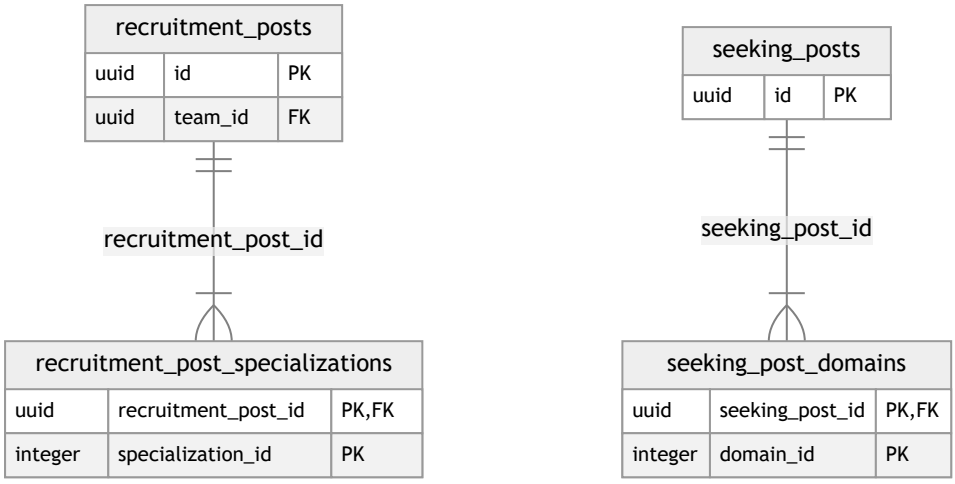
**FK** fk_join_request_ideas_request FOREIGN KEY (join_request_id) REFERENCES team.join_requests(id) ON DELETE CASCADE

**CHECK** ck_join_request_ideas_position CHECK ((("position" >= 1) AND ("position" <= 3)))

**CHECK** ck_join_request_ideas_title CHECK ((char_length(btrim(title)) >= 1))

**TRIGGER** trg_join_request_ideas_check AFTER INSERT OR DELETE ON team.join_request_ideas DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION team.tg_join_requests_ideas()

## Bài tuyển thành viên, bài tìm nhóm




**team.recruitment_posts**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| team_id | uuid | | FK | | |
| title | varchar(150) | | | | |
| description | varchar(2000) | ✓ | | | |
| status | varchar(10) | | | OPEN | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |
| closed_at | timestamptz | ✓ | | | |

**PK** pk_recruitment_posts PRIMARY KEY (id)

**FK** fk_recruitment_posts_team FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE CASCADE

**CHECK** ck_recruitment_posts_closed CHECK ((((status) = 'CLOSED') = (closed_at IS NOT NULL)))

**CHECK** ck_recruitment_posts_status CHECK ((status IN ('OPEN', 'CLOSED')))

**UNIQUE INDEX** uq_recruitment_posts_open btree (team_id) WHERE ((status) = 'OPEN')

**TRIGGER** trg_recruitment_posts_updated_at BEFORE UPDATE ON team.recruitment_posts FOR EACH ROW EXECUTE FUNCTION team.set_updated_at()

**team.recruitment_post_specializations**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| recruitment_post_id | uuid | | PK FK | | |
| specialization_id | integer | | PK | | |

**PK** pk_recruitment_post_specializations PRIMARY KEY (recruitment_post_id, specialization_id)

**FK** fk_rps_post FOREIGN KEY (recruitment_post_id) REFERENCES team.recruitment_posts(id) ON DELETE CASCADE

**INDEX** idx_rps_specialization_id btree (specialization_id)

**team.seeking_posts**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| semester_id | uuid | | | | |
| student_id | uuid | | | | |
| specialization_id | integer | | | | |
| title | varchar(150) | | | | |
| description | varchar(2000) | ✓ | | | |
| status | varchar(10) | | | OPEN | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |
| closed_at | timestamptz | ✓ | | | |

**PK** pk_seeking_posts PRIMARY KEY (id)
**CHECK** ck_seeking_posts_closed CHECK ((((status) = 'CLOSED') = (closed_at IS NOT NULL)))
**CHECK** ck_seeking_posts_status CHECK ((status IN ('OPEN', 'CLOSED')))
**INDEX** idx_seeking_posts_semester_spec btree (semester_id, specialization_id) WHERE ((status) = 'OPEN')
**UNIQUE INDEX** uq_seeking_posts_open btree (semester_id, student_id) WHERE ((status) = 'OPEN')
**TRIGGER** trg_seeking_posts_updated_at BEFORE UPDATE ON team.seeking_posts FOR EACH ROW EXECUTE FUNCTION team.set_updated_at()

**team.seeking_post_domains**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| seeking_post_id | uuid | | PK FK | | |
| domain_id | integer | | PK | | |
| domain_code | varchar(3) | | | | |

**PK** pk_seeking_post_domains PRIMARY KEY (seeking_post_id, domain_id)
**FK** fk_spd_post FOREIGN KEY (seeking_post_id) REFERENCES team.seeking_posts(id) ON DELETE CASCADE
**CHECK** ck_spd_code CHECK (((domain_code) ~ '^D(0[1-9]|1[0-8])$'))

## Yêu cầu đổi nhóm, chỉ EXE201 (rule 7)

**team.change_requests**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| team_id | uuid | | FK | | |
| course_code | varchar(6) | | FK | EXE201 | |
| round_id | uuid | | | | academic.registration_rounds.id (đợt đổi nhóm) |
| requested_by_student_id | uuid | | | | |
| reason | varchar(1000) | | | | |
| changed_member_count | smallint | | | | hệ thống tự tính so với mốc EXE101 |
| changed_percent | numeric(5,2) | | | | |
| status | varchar(10) | | | PENDING | |
| reviewed_by_account_id | uuid | ✓ | | | Admin (identity) |
| reviewed_at | timestamptz | ✓ | | | |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| review_note | varchar(1000) | ✓ | | | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**PK** pk_change_requests PRIMARY KEY (id)

**FK** fk_change_requests_team FOREIGN KEY (team_id, course_code) REFERENCES team.teams(id, course_code) ON DELETE RESTRICT

**CHECK** ck_change_requests_counts CHECK (((changed_member_count >= 0) AND ((changed_percent >= (0)::numeric) AND (changed_percent <= (100)::numeric))))

**CHECK** ck_change_requests_course CHECK (((course_code) = 'EXE201'))

**CHECK** ck_change_requests_reason CHECK ((char_length(btrim(reason)) >= 10))

**CHECK** ck_change_requests_reject_note CHECK ((((status) <> 'REJECTED') OR (review_note IS NOT NULL)))

**CHECK** ck_change_requests_review CHECK (((status IN ('APPROVED', 'REJECTED')) = ((reviewed_by_account_id IS NOT NULL) AND (reviewed_at IS NOT NULL))))

**CHECK** ck_change_requests_status CHECK ((status IN ('PENDING', 'APPROVED', 'REJECTED', 'CANCELLED')))

**INDEX** idx_change_requests_round btree (round_id, status)

**INDEX** idx_change_requests_status_created btree (status, created_at DESC)

**UNIQUE INDEX** uq_change_requests_pending btree (team_id) WHERE ((status) = 'PENDING')

**TRIGGER** trg_change_requests_updated_at BEFORE UPDATE ON team.change_requests FOR EACH ROW EXECUTE FUNCTION team.set_updated_at()

#### team.change_request_items

Thay đổi chờ duyệt của nhóm kế thừa. change_request_id NULL = đã được hai bên đồng ý nhưng chưa đưa vào yêu cầu.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| team_id | uuid | | FK | | |
| change_request_id | uuid | ✓ | FK | | |
| action | varchar(14) | | | | |
| source | varchar(20) | | | | |
| student_id | uuid | ✓ | | | NULL khi PROJECT_CHANGE |
| student_code | varchar(8) | ✓ | | | |
| full_name | varchar(150) | ✓ | | | |
| specialization_id | integer | ✓ | | | |
| new_project_name | varchar(200) | ✓ | | | PROJECT_CHANGE (rule 26) |
| new_project_description | varchar(4000) | ✓ | | | |
| created_at | timestamptz | | | now() | |

**PK** pk_change_request_items PRIMARY KEY (id)

**FK** fk_change_request_items_request FOREIGN KEY (change_request_id) REFERENCES team.change_requests(id) ON DELETE CASCADE

**FK** fk_change_request_items_team FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE CASCADE

**CHECK** ck_change_request_items_action CHECK ((action IN ('ADD', 'REMOVE', 'SET_LEADER', 'PROJECT_CHANGE')))

**CHECK** ck_change_request_items_code CHECK (((student_code IS NULL) OR ((student_code) ~ '^[A-Z]{2}[0-9]{6}$')))

**CHECK** ck_change_request_items_source CHECK ((source IN ('INVITATION', 'JOIN_REQUEST', 'LEAVE', 'REMOVE', 'LEADER_TRANSFER', 'PROJECT')))

**CHECK** ck_change_request_items_spec CHECK ((((action) <> 'ADD') OR (specialization_id IS NOT NULL)))

**CHECK** ck_change_request_items_target CHECK (((((action) = 'PROJECT_CHANGE') AND ((source) = 'PROJECT') AND (student_id IS NULL) AND (new_project_name IS NOT NULL)) OR (((action) <> 'PROJECT_CHANGE') AND ((source) <> 'PROJECT') AND (student_id IS NOT NULL) AND (student_code IS NOT NULL) AND (full_name IS NOT NULL) AND (new_project_name IS NULL))))

**INDEX** idx_change_request_items_student_id btree (student_id)

**UNIQUE INDEX** uq_change_request_items_pending btree (team_id, student_id, action) WHERE (change_request_id IS NULL)

**UNIQUE INDEX** uq_change_request_items_project_pending btree (team_id) WHERE (((action) = 'PROJECT_CHANGE') AND (change_request_id IS NULL))

**UNIQUE INDEX** uq_change_request_items_request btree (change_request_id, student_id, action) WHERE (change_request_id IS NOT NULL)

## Quan tâm của cố vấn (rule 19)

**team.topic_interests**

Giảng viên hoặc mentor bày tỏ muốn nhận nhóm. Không giữ chỗ; là một đầu vào của ghép tự động (rule 9, 20). Giới hạn số quan tâm kiểm tra ở Services. CLOSED khi nhóm đã được ghép cố vấn cùng loại, WITHDRAWN khi người đó tự rút.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| team_id | uuid | | FK | | |
| advisor_kind | varchar(10) | | | | LECTURER \| MENTOR |
| advisor_id | uuid | | | | academic.lecturers.id hoặc academic.mentors.id |
| advisor_name | varchar(150) | | | | snapshot |
| status | varchar(10) | | | OPEN | |
| closed_reason | varchar(30) | ✓ | | | |
| created_at | timestamptz | | | now() | |
| closed_at | timestamptz | ✓ | | | |

**PK** pk_topic_interests PRIMARY KEY (id)

**FK** fk_topic_interests_team FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE CASCADE

**CHECK** ck_topic_interests_closed CHECK ((((status) = 'OPEN') = (closed_at IS NULL)))

**CHECK** ck_topic_interests_kind CHECK ((advisor_kind IN ('LECTURER', 'MENTOR')))

**CHECK** ck_topic_interests_reason CHECK (((((status) = 'CLOSED') = (closed_reason IS NOT NULL)) AND ((closed_reason IS NULL) OR (closed_reason IN ('THIS_ADVISOR_MATCHED', 'OTHER_ADVISOR_MATCHED', 'TEAM_DISBANDED')))))

**CHECK** ck_topic_interests_status CHECK ((status IN ('OPEN', 'WITHDRAWN', 'CLOSED')))

**INDEX** idx_topic_interests_advisor btree (advisor_kind, advisor_id, status)

**UNIQUE INDEX** uq_topic_interests_open btree (team_id, advisor_kind, advisor_id) WHERE ((status) = 'OPEN')

## Nguyện vọng giảng viên của nhóm (rule 9)

**team.lecturer_preferences**

Tối đa 3 (rule_settings.max_lecturer_preferences, kiểm tra ở Services), có thứ tự. Không có trạng thái: trưởng nhóm sửa được tới khi nhóm được ghép giảng viên, sau đó khóa (Services kiểm tra). Không còn bước chấp nhận / từ chối.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| team_id | uuid | | UQ PK FK | | |
| rank | smallint | | PK | | |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| lecturer_id | uuid | | UQ | | academic.lecturers.id |
| lecturer_name | varchar(150) | | | | snapshot |
| updated_by_student_id | uuid | | | | |
| updated_at | timestamptz | | | now() | |

**UNIQUE** uq_lecturer_preferences_lecturer UNIQUE (team_id, lecturer_id)

**PK** pk_lecturer_preferences PRIMARY KEY (team_id, rank)

**FK** fk_lecturer_preferences_team FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE CASCADE

**CHECK** ck_lecturer_preferences_rank CHECK (((rank >= 1) AND (rank <= 10)))

**INDEX** idx_lecturer_preferences_lecturer btree (lecturer_id)

## Lần chạy ghép cố vấn (rule 9, 20)

**team.advisor_matching_runs**

Mỗi lần ghép (tự động lúc hết hạn / tuần mentor, hoặc Admin bấm "Ghép sớm") là một dòng. PREVIEW: kết quả xem trước nằm trong plan; APPLIED: đã ghi vào *_assignments; DISCARDED: Admin bỏ.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| advisor_kind | varchar(10) | | | | |
| semester_id | uuid | | | | |
| course_code | varchar(6) | | | | |
| trigger | varchar(10) | | | | |
| status | varchar(10) | | | PREVIEW | |
| requested_by_account_id | uuid | ✓ | | | Admin khi Ghép sớm; NULL khi Job chạy |
| plan | jsonb | | | [] | [{teamId, advisorId, reason, preferenceRank}] |
| matched_count | integer | | | 0 | |
| unmatched_count | integer | | | 0 | |
| created_at | timestamptz | | | now() | |
| applied_at | timestamptz | ✓ | | | |

**PK** pk_advisor_matching_runs PRIMARY KEY (id)

**CHECK** ck_advisor_matching_runs_applied CHECK ((((status) = 'APPLIED') = (applied_at IS NOT NULL)))

**CHECK** ck_advisor_matching_runs_course CHECK ((course_code IN ('EXE101', 'EXE201')))

**CHECK** ck_advisor_matching_runs_kind CHECK ((advisor_kind IN ('LECTURER', 'MENTOR')))

**CHECK** ck_advisor_matching_runs_requester CHECK ((((trigger) = 'SCHEDULED') OR (requested_by_account_id IS NOT NULL)))

**CHECK** ck_advisor_matching_runs_status CHECK ((status IN ('PREVIEW', 'APPLIED', 'DISCARDED')))

**CHECK** ck_advisor_matching_runs_trigger CHECK ((trigger IN ('SCHEDULED', 'EARLY')))

**INDEX** idx_advisor_matching_runs_semester btree (semester_id, course_code, advisor_kind)

## Giảng viên hướng dẫn của nhóm (rule 9)

**team.lecturer_assignments**

Một giảng viên hướng dẫn đang hiệu lực mỗi nhóm. reason là nhãn hiện cho nhóm, giảng viên và Admin.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| team_id | uuid | | FK | | |
| lecturer_id | uuid | | | | |
| lecturer_name | varchar(150) | | | | snapshot |
| status | varchar(10) | | | ASSIGNED | |
| reason | varchar(20) | | | | |
| preference_rank | smallint | ✓ | | | với MUTUAL / PREFERENCE: nguyện vọng thứ mấy |
| source | varchar(10) | | | AUTO | |
| matching_run_id | uuid | ✓ | FK | | |
| assigned_by_account_id | uuid | ✓ | | | Admin khi MANUAL |
| assigned_at | timestamptz | | | now() | |
| revoked_at | timestamptz | ✓ | | | |
| note | varchar(500) | ✓ | | | lý do khi Admin đổi tay |

**PK** pk_lecturer_assignments PRIMARY KEY (id)

**FK** fk_lecturer_assignments_run FOREIGN KEY (matching_run_id) REFERENCES team.advisor_matching_runs(id) ON DELETE SET NULL

**FK** fk_lecturer_assignments_team FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE RESTRICT

**CHECK** ck_lecturer_assignments_rank CHECK (((reason IN ('MUTUAL', 'PREFERENCE')) = (preference_rank IS NOT NULL)))

**CHECK** ck_lecturer_assignments_reason CHECK ((reason IN ('MUTUAL', 'PREFERENCE', 'LECTURER_INTEREST', 'PREVIOUS_INSTRUCTOR', 'DOMAIN', 'MANUAL')))

**CHECK** ck_lecturer_assignments_revoked CHECK ((((status) = 'REVOKED') = (revoked_at IS NOT NULL)))

**CHECK** ck_lecturer_assignments_source CHECK (((((source) = 'AUTO') AND ((reason) <> 'MANUAL')) OR (((source) = 'MANUAL') AND ((reason) = 'MANUAL') AND (assigned_by_account_id IS NOT NULL) AND (note IS NOT NULL))))

**CHECK** ck_lecturer_assignments_status CHECK ((status IN ('ASSIGNED', 'REVOKED')))

**INDEX** idx_lecturer_assignments_lecturer btree (lecturer_id, status)

**UNIQUE INDEX** uq_lecturer_assignments_active btree (team_id) WHERE ((status) = 'ASSIGNED')

## Mentor của nhóm (rule 20)

**team.mentor_assignments**

Một mentor đang hiệu lực mỗi nhóm; ghép tự động ở tuần bắt đầu mentor theo quan tâm của mentor.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| team_id | uuid | | FK | | |
| mentor_id | uuid | | | | academic.mentors.id |
| mentor_name | varchar(150) | | | | snapshot |
| status | varchar(10) | | | ASSIGNED | |
| reason | varchar(20) | | | | |
| source | varchar(10) | | | AUTO | |
| matching_run_id | uuid | ✓ | FK | | |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| assigned_by_account_id | uuid | ✓ | | | Admin khi MANUAL |
| assigned_at | timestamptz | | | now() | |
| ended_at | timestamptz | ✓ | | | |
| end_reason | varchar(500) | ✓ | | | |
| note | varchar(500) | ✓ | | | lý do khi Admin đổi tay |

**PK** pk_mentor_assignments PRIMARY KEY (id)

**FK** fk_mentor_assignments_run FOREIGN KEY (matching_run_id) REFERENCES team.advisor_matching_runs(id) ON DELETE SET NULL

**FK** fk_mentor_assignments_team FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE RESTRICT

**CHECK** ck_mentor_assignments_ended CHECK ((((status) = 'ENDED') = (ended_at IS NOT NULL)))

**CHECK** ck_mentor_assignments_reason CHECK ((reason IN ('MENTOR_INTEREST', 'DOMAIN', 'MANUAL')))

**CHECK** ck_mentor_assignments_source CHECK (((((source) = 'AUTO') AND ((reason) <> 'MANUAL')) OR (((source) = 'MANUAL') AND ((reason) = 'MANUAL') AND (assigned_by_account_id IS NOT NULL) AND (note IS NOT NULL))))

**CHECK** ck_mentor_assignments_status CHECK ((status IN ('ASSIGNED', 'ENDED')))

**INDEX** idx_mentor_assignments_mentor btree (mentor_id, status)

**UNIQUE INDEX** uq_mentor_assignments_active btree (team_id) WHERE ((status) = 'ASSIGNED')

## Nhật ký (rule 10), Outbox

**team.audit_logs**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | bigint | | PK | IDENTITY | |
| team_id | uuid | ✓ | | | |
| entity_type | varchar(50) | | | | |
| entity_id | uuid | | | | |
| action | varchar(50) | | | | |
| actor_account_id | uuid | ✓ | | | NULL = hệ thống / job |
| actor_role | varchar(20) | | | | |
| actor_name | varchar(150) | ✓ | | | snapshot tên người thực hiện (S22, W24); NULL = hệ thống / log cũ |
| old_values | jsonb | ✓ | | | |
| new_values | jsonb | ✓ | | | |
| reason | varchar(1000) | ✓ | | | |
| correlation_id | varchar(64) | ✓ | | | |
| occurred_at | timestamptz | | | now() | |

**PK** pk_audit_logs PRIMARY KEY (id)

**CHECK** ck_audit_logs_actor_role CHECK ((actor_role IN ('Student', 'Lecturer', 'Mentor', 'Admin', 'System')))

**INDEX** idx_audit_logs_actor btree (actor_account_id)

**INDEX** idx_audit_logs_entity btree (entity_type, entity_id)

**INDEX** idx_audit_logs_team_time btree (team_id, occurred_at DESC)

**TRIGGER** trg_audit_logs_append_only BEFORE DELETE OR UPDATE ON team.audit_logs FOR EACH ROW EXECUTE FUNCTION team.tg_audit_logs_append_only()

**team.outbox_messages**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | | = eventId của envelope |
| topic | varchar(100) | | | | |
| event_type | varchar(100) | | | | |
| aggregate_type | varchar(50) | | | | |
| aggregate_id | uuid | | | | Kafka message key |
| payload | jsonb | | | | |
| correlation_id | varchar(64) | ✓ | | | |
| occurred_at | timestamptz | | | now() | |
| processed_at | timestamptz | ✓ | | | |
| attempts | integer | | | 0 | |
| last_error | text | ✓ | | | |

**PK** pk_outbox_messages PRIMARY KEY (id)

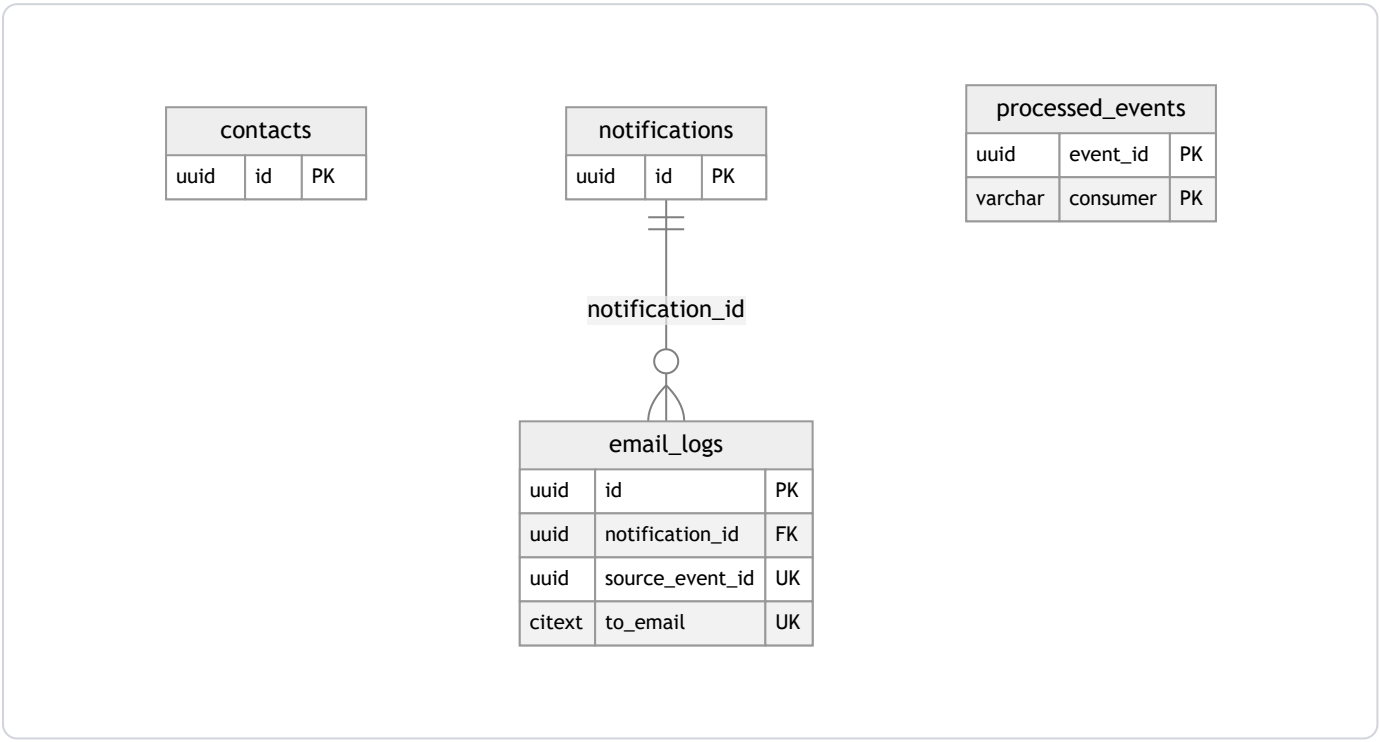
**CHECK** ck_outbox_messages_attempts CHECK (attempts >= 0)

**INDEX** idx_outbox_messages_pending btree (occurred_at) WHERE (processed_at IS NULL)

# 7. Schema notification — Notification Service

Database `notification_db`, schema `notification`, login role `notification_svc` · 4 bảng · script `deploy/db/04_notification_db.sql`

Read model người nhận (`contacts`) dựng từ `exe.student`; thông báo trong ứng dụng và log email. Idempotent theo eventId.




**`notification.contacts`**

Danh bạ người nhận (email, tên) dựng từ topic exe.student (StudentImported, StudentProfileUpdated, LecturerRegistered, MentorRegistered).

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| `id` | `uuid` | | PK | | academic students.id / lecturers.id / mentors.id |
| `person_type` | `varchar(10)` | | | | |
| `email` | `citext` | | | | |
| `full_name` | `varchar(150)` | | | | |
| `email_enabled` | `boolean` | | | `true` | công tắc nhận email thông báo (A07) |
| `updated_at` | `timestamptz` | | | `now()` | |

**PK** `pk_contacts` `PRIMARY KEY (id)`

**CHECK** `ck_contacts_person_type` `CHECK ((person_type IN ('STUDENT', 'LECTURER', 'MENTOR')))`

**INDEX** `idx_contacts_email` `btree (email)`

**`notification.notifications`**

Thông báo trong ứng dụng. ADMIN -> recipient_id NULL (thông báo chung). UNIQUE (sự kiện, người nhận) để idempotent.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| `id` | `uuid` | | PK | `gen_random_uuid()` | |
| `recipient_type` | `varchar(10)` | | | | |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| recipient_id | uuid | ✓ | | | |
| type | varchar(50) | | | | = eventType |
| title | varchar(200) | | | | |
| content | varchar(2000) | | | | |
| link_url | varchar(500) | ✓ | | | màn hình cần mở khi chạm |
| source_event_id | uuid | | | | |
| is_read | boolean | | | false | |
| read_at | timestamptz | ✓ | | | |
| created_at | timestamptz | | | now() | |

**PK** pk_notifications PRIMARY KEY (id)

**CHECK** ck_notifications_read CHECK ((is_read = (read_at IS NOT NULL)))

**CHECK** ck_notifications_recipient CHECK ((((recipient_type) = 'ADMIN') = (recipient_id IS NULL)))

**CHECK** ck_notifications_recipient_type CHECK ((recipient_type IN ('STUDENT', 'LECTURER', 'MENTOR', 'ADMIN')))

**INDEX** idx_notifications_recipient btree (recipient_type, recipient_id, created_at DESC)

**INDEX** idx_notifications_unread btree (recipient_type, recipient_id) WHERE (NOT is_read)

**UNIQUE INDEX** uq_notifications_event_recipient btree (source_event_id, recipient_type, recipient_id) NULLS NOT DISTINCT

#### notification.email_logs

Nhật ký gửi email SMTP; retry khi FAILED.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| notification_id | uuid | ✓ | FK | | |
| source_event_id | uuid | | UQ | | |
| to_email | citext | | UQ | | |
| subject | varchar(255) | | | | |
| template_code | varchar(50) | | | | |
| status | varchar(10) | | | PENDING | |
| attempts | integer | | | 0 | |
| last_error | text | ✓ | | | |
| created_at | timestamptz | | | now() | |
| sent_at | timestamptz | ✓ | | | |

**UNIQUE** uq_email_logs_event_to UNIQUE (source_event_id, to_email)

**PK** pk_email_logs PRIMARY KEY (id)

**FK** fk_email_logs_notification FOREIGN KEY (notification_id) REFERENCES notification.notifications(id) ON DELETE SET NULL

**CHECK** ck_email_logs_attempts CHECK (attempts >= 0)

**CHECK** ck_email_logs_sent CHECK ((((status) = 'SENT') = (sent_at IS NOT NULL)))

**CHECK** ck_email_logs_status CHECK ((status IN ('PENDING', 'SENT', 'FAILED')))

**INDEX** idx_email_logs_notification_id btree (notification_id)

**INDEX** idx_email_logs_retry btree (created_at) WHERE (status IN ('PENDING', 'FAILED'))

**notification.processed_events**

EventId đã xử lý theo từng handler, chống gửi trùng.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| event_id | uuid | | PK | | |
| consumer | varchar(100) | | PK | | |
| topic | varchar(100) | | | | |
| event_type | varchar(100) | | | | |
| processed_at | timestamptz | | | now() | |

**PK** pk_processed_events PRIMARY KEY (event_id, consumer)

**INDEX** idx_processed_events_processed_at btree (processed_at)

# 8. Schema collab — Collaboration Service

Database `collab_db`, schema `collab`, login role `collab_svc` · 27 bảng · script `deploy/db/06_collab_db.sql`

Workspace của nhóm (thêm 19/10/2026): tin nhắn, file, việc và bài nộp, họp, checkpoint. `workspace_members` là read model dựng từ `exe.team` và `exe.assignment`; file nhị phân nằm ở S3 (bucket `exe-files`). Publish qua `outbox_messages` (topic `exe.collab`).

## Workspace và thành viên (rule 21)

**`collab.workspaces`**

Một workspace cho mỗi nhóm, tạo khi consume TeamCreated. previous_workspace_id = workspace EXE101 cùng đề tài (rule 26).

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| team_id | uuid | | UQ | | team.teams.id |
| project_id | uuid | ✓ | | | team.projects.id |
| semester_id | uuid | | | | |
| course_code | varchar(6) | | | | |
| team_name | varchar(100) | | | | snapshot |
| previous_workspace_id | uuid | ✓ | FK | | |
| created_at | timestamptz | | | now() | |
| archived_at | timestamptz | ✓ | | | |

**UNIQUE** `uq_workspaces_team` `UNIQUE (team_id)`

**PK** `pk_workspaces` `PRIMARY KEY (id)`

**FK** `fk_workspaces_previous` `FOREIGN KEY (previous_workspace_id) REFERENCES collab.workspaces(id) ON DELETE SET NULL`

**CHECK** `ck_workspaces_course` `CHECK ((course_code IN ('EXE101', 'EXE201')))`

**CHECK** `ck_workspaces_previous_self` `CHECK (((previous_workspace_id IS NULL) OR (previous_workspace_id <> id)))`

**INDEX** `idx_workspaces_previous` `btree (previous_workspace_id)`

**INDEX** `idx_workspaces_project` `btree (project_id)`

**`collab.workspace_members`**

Read model quyền truy cập, dựng từ Kafka (MemberJoined, MemberLeft, LeaderChanged, LecturerAssigned, MentorAssigned...). Collaboration không gọi Team để kiểm tra quyền. Dòng không bị xóa: rời nhóm thì ghi left_at.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| workspace_id | uuid | | PK FK | | |
| person_type | varchar(10) | | PK | | |
| person_id | uuid | | PK | | |
| display_name | varchar(150) | | | | |
| role | varchar(10) | | | | |
| joined_at | timestamptz | | | now() | |
| left_at | timestamptz | ✓ | | | |

**PK** `pk_workspace_members` `PRIMARY KEY (workspace_id, person_type, person_id)`

**FK** fk_workspace_members_workspace FOREIGN KEY (workspace_id) REFERENCES collab.workspaces(id) ON DELETE CASCADE

**CHECK** ck_workspace_members_left CHECK (((left_at IS NULL) OR (left_at >= joined_at)))

**CHECK** ck_workspace_members_person_type CHECK ((person_type IN ('STUDENT', 'LECTURER', 'MENTOR')))

**CHECK** ck_workspace_members_role CHECK ((role IN ('LEADER', 'MEMBER', 'INSTRUCTOR', 'MENTOR')))

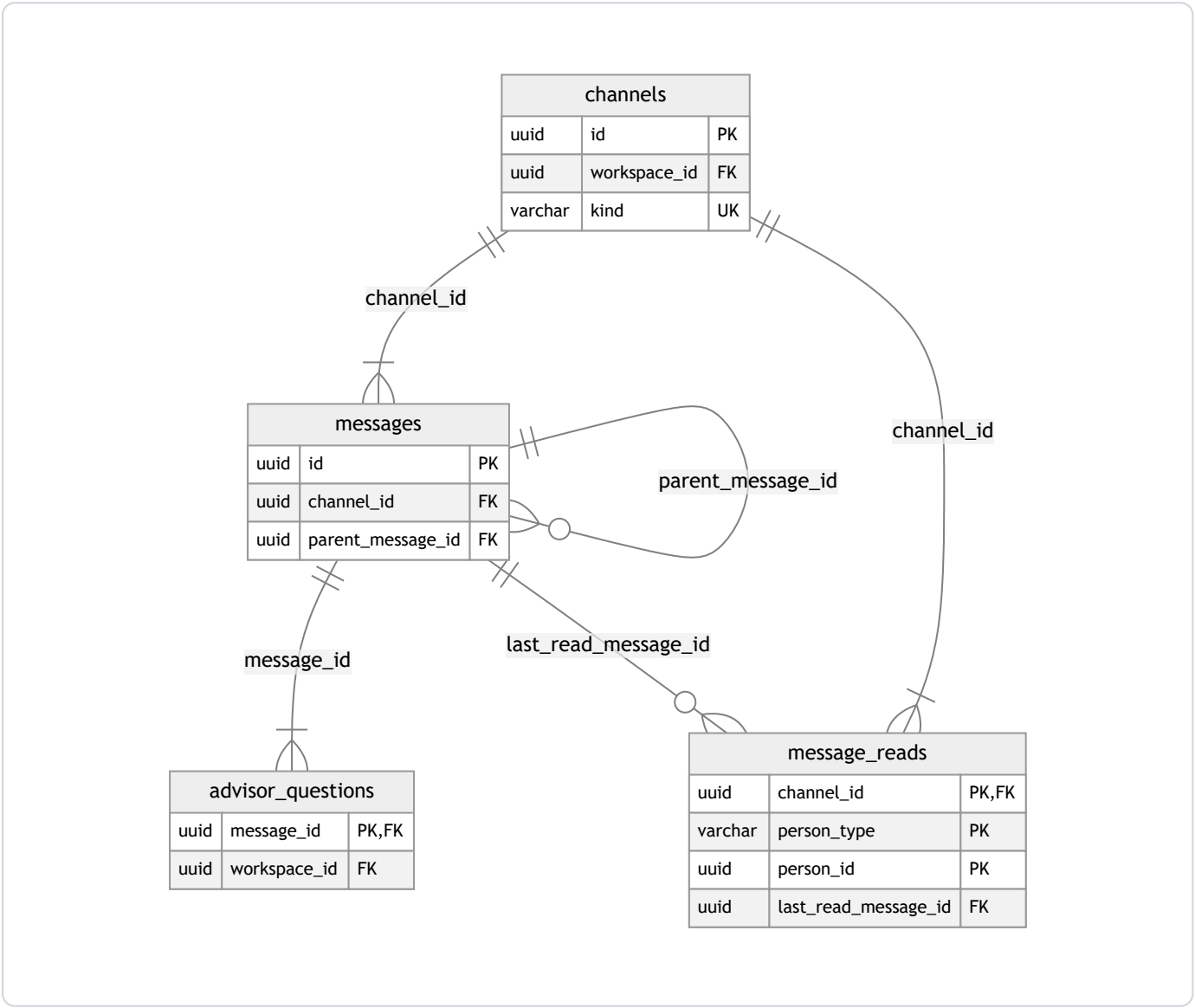
**CHECK** ck_workspace_members_role_type CHECK ((((role IN ('LEADER', 'MEMBER')) AND ((person_type) = 'STUDENT')) OR (((role) = 'INSTRUCTOR') AND ((person_type) = 'LECTURER')) OR (((role) = 'MENTOR') AND (person_type IN ('MENTOR', 'LECTURER')))))

**INDEX** idx_workspace_members_person btree (person_type, person_id) WHERE (left_at IS NULL)

**UNIQUE INDEX** uq_workspace_members_one_instructor btree (workspace_id) WHERE (((role) = 'INSTRUCTOR') AND (left_at IS NULL))

**UNIQUE INDEX** uq_workspace_members_one_mentor btree (workspace_id) WHERE (((role) = 'MENTOR') AND (left_at IS NULL))

## Tin nhắn (rule 22)




**collab.channels**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| workspace_id | uuid | | UQ FK | | |
| kind | varchar(10) | | UQ | | |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| created_at | timestamptz | | | now() | |

**UNIQUE** uq_channels_workspace_kind UNIQUE (workspace_id, kind)

**PK** pk_channels PRIMARY KEY (id)

**FK** fk_channels_workspace FOREIGN KEY (workspace_id) REFERENCES collab.workspaces(id) ON DELETE CASCADE

**CHECK** ck_channels_kind CHECK ((kind IN ('TEAM', 'ADVISORS')))

**collab.messages**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| channel_id | uuid | | FK | | |
| parent_message_id | uuid | ✓ | FK | | trả lời trong luồng |
| author_person_type | varchar(10) | | | | |
| author_person_id | uuid | | | | |
| author_name | varchar(150) | | | | snapshot |
| body | varchar(2000) | | | | |
| is_advisor_question | boolean | | | false | |
| created_at | timestamptz | | | now() | |
| edited_at | timestamptz | ✓ | | | |
| deleted_at | timestamptz | ✓ | | | xóa mềm, hiển thị "Tin nhắn đã bị xóa" |

**PK** pk_messages PRIMARY KEY (id)

**FK** fk_messages_channel FOREIGN KEY (channel_id) REFERENCES collab.channels(id) ON DELETE CASCADE

**FK** fk_messages_parent FOREIGN KEY (parent_message_id) REFERENCES collab.messages(id) ON DELETE CASCADE

**CHECK** ck_messages_author_type CHECK ((author_person_type IN ('STUDENT', 'LECTURER', 'MENTOR')))

**CHECK** ck_messages_body CHECK ((char_length(btrim(body)) >= 1))

**CHECK** ck_messages_question_root CHECK (((NOT is_advisor_question) OR (parent_message_id IS NULL)))

**INDEX** idx_messages_channel_cursor btree (channel_id, created_at DESC, id DESC) WHERE (parent_message_id IS NULL)

**INDEX** idx_messages_thread btree (parent_message_id, created_at) WHERE (parent_message_id IS NOT NULL)

**collab.advisor_questions**

Câu hỏi cho cố vấn: OPEN tới khi giảng viên hoặc mentor trả lời trong luồng.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| message_id | uuid | | PK FK | | |
| workspace_id | uuid | | FK | | |
| status | varchar(10) | | | OPEN | |
| answered_by_person_type | varchar(10) | ✓ | | | |
| answered_by_person_id | uuid | ✓ | | | |
| answered_at | timestamptz | ✓ | | | |

**PK** pk_advisor_questions PRIMARY KEY (message_id)

**FK** fk_advisor_questions_message FOREIGN KEY (message_id) REFERENCES collab.messages(id) ON DELETE CASCADE

**FK** fk_advisor_questions_workspace FOREIGN KEY (workspace_id) REFERENCES collab.workspaces(id) ON DELETE CASCADE

**CHECK** ck_advisor_questions_answered CHECK ((((status) = 'ANSWERED') = ((answered_at IS NOT NULL) AND (answered_by_person_id IS NOT NULL))))

**CHECK** ck_advisor_questions_answerer CHECK (((answered_by_person_type IS NULL) OR (answered_by_person_type IN ('LECTURER', 'MENTOR'))))

**CHECK** ck_advisor_questions_status CHECK ((status IN ('OPEN', 'ANSWERED')))

**INDEX** idx_advisor_questions_open btree (workspace_id) WHERE ((status) = 'OPEN')

**collab.message_reads**

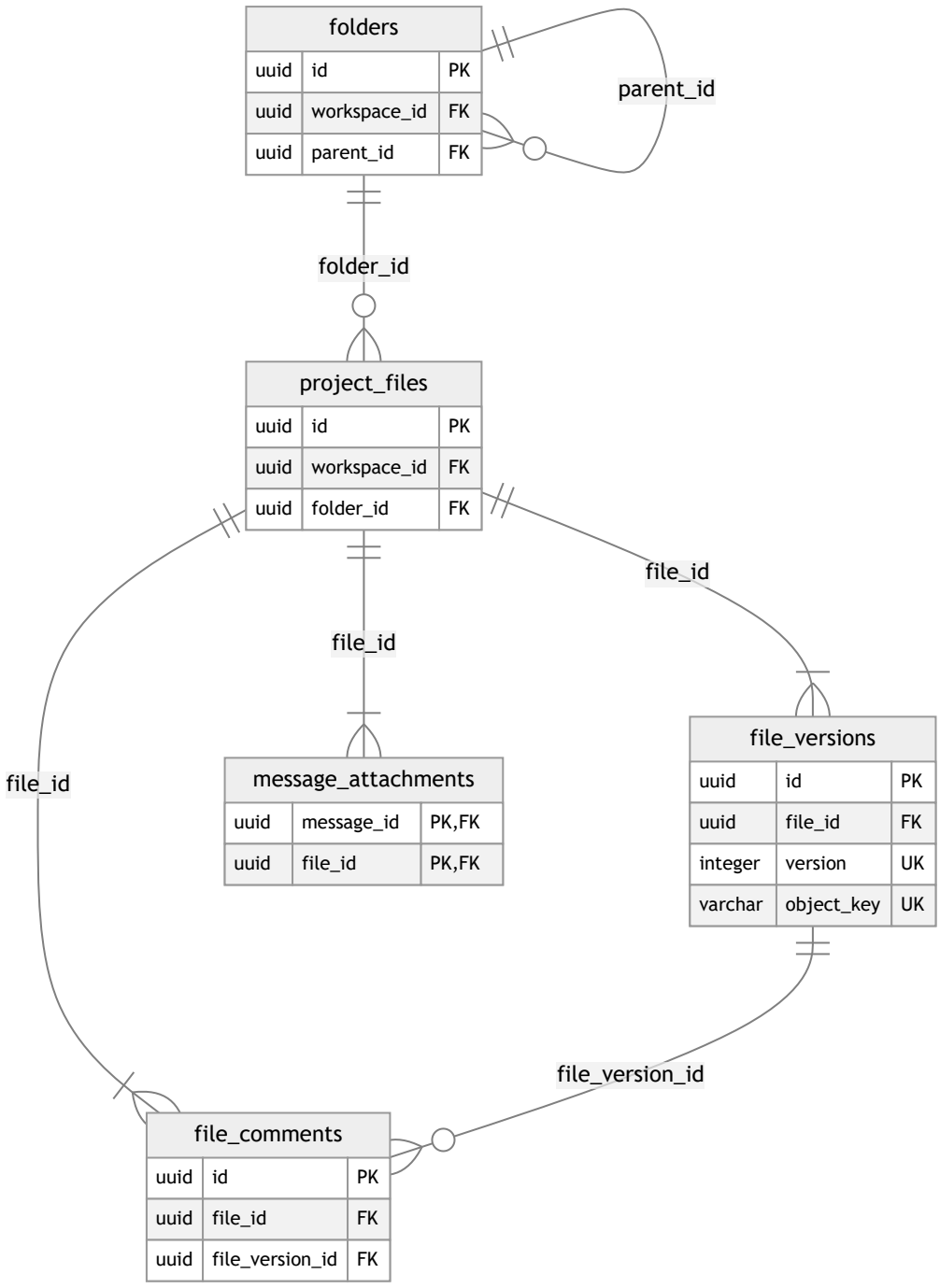
| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| channel_id | uuid | | PK FK | | |
| person_type | varchar(10) | | PK | | |
| person_id | uuid | | PK | | |
| last_read_message_id | uuid | ✓ | FK | | |
| last_read_at | timestamptz | | | now() | |

**PK** pk_message_reads PRIMARY KEY (channel_id, person_type, person_id)

**FK** fk_message_reads_channel FOREIGN KEY (channel_id) REFERENCES collab.channels(id) ON DELETE CASCADE

**FK** fk_message_reads_message FOREIGN KEY (last_read_message_id) REFERENCES collab.messages(id) ON DELETE SET NULL

## File dự án (rule 24)




**`collab.folders`**

Thư mục is_archive = "EXE101 (kỳ trước)": chỉ đọc. Nội dung file nằm ở MinIO bucket exe-files.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| `id` | `uuid` | | PK | `gen_random_uuid()` | |
| `workspace_id` | `uuid` | | FK | | |
| `parent_id` | `uuid` | ✓ | FK | | |
| `name` | `varchar(100)` | | | | |
| `is_archive` | `boolean` | | | `false` | |
| `created_at` | `timestamptz` | | | `now()` | |

**PK** pk_folders PRIMARY KEY (id)

**FK** fk_folders_parent FOREIGN KEY (parent_id) REFERENCES collab.folders(id) ON DELETE CASCADE

**FK** fk_folders_workspace FOREIGN KEY (workspace_id) REFERENCES collab.workspaces(id) ON DELETE CASCADE

**CHECK** ck_folders_name CHECK ((char_length(btrim(name)) >= 1))

**UNIQUE INDEX** uq_folders_name btree (workspace_id, COALESCE(parent_id, '00000000-0000-0000-0000-000000000000'::uuid), lower(name))

**collab.project_files**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| workspace_id | uuid | | FK | | |
| folder_id | uuid | ✓ | FK | | NULL = thư mục gốc |
| name | varchar(255) | | | | |
| current_version | integer | | | 1 | |
| created_by_type | varchar(10) | | | | |
| created_by_id | uuid | | | | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |
| deleted_at | timestamptz | ✓ | | | xóa mềm (người tải lên, trưởng nhóm) |

**PK** pk_project_files PRIMARY KEY (id)

**FK** fk_project_files_folder FOREIGN KEY (folder_id) REFERENCES collab.folders(id) ON DELETE CASCADE

**FK** fk_project_files_workspace FOREIGN KEY (workspace_id) REFERENCES collab.workspaces(id) ON DELETE CASCADE

**CHECK** ck_project_files_creator CHECK (((created_by_type) = 'STUDENT'))

**CHECK** ck_project_files_type CHECK (((name) ~* '\.(pdf|docx|pptx|xlsx|png|jpe?g|zip)$'))

**CHECK** ck_project_files_version CHECK (current_version >= 1)

**UNIQUE INDEX** uq_project_files_name btree (workspace_id, COALESCE(folder_id, '00000000-0000-0000-0000-000000000000'::uuid), lower(name)) WHERE (deleted_at IS NULL)

**TRIGGER** trg_project_files_updated_at BEFORE UPDATE ON collab.project_files FOR EACH ROW EXECUTE FUNCTION collab.set_updated_at()

**collab.file_versions**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| file_id | uuid | | UQ FK | | |
| version | integer | | UQ | | |
| object_key | varchar(500) | | UQ | | workspace/{id}/{fileId}/{version} |
| size_bytes | bigint | | | | |
| content_type | varchar(150) | | | | |
| uploaded_by_id | uuid | | | | students.id |
| uploaded_by_name | varchar(150) | | | | |
| created_at | timestamptz | | | now() | |

**UNIQUE** uq_file_versions_file_version UNIQUE (file_id, version)

**UNIQUE** uq_file_versions_object_key UNIQUE (object_key)

**PK** pk_file_versions PRIMARY KEY (id)

**FK** fk_file_versions_file FOREIGN KEY (file_id) REFERENCES collab.project_files(id) ON DELETE CASCADE

**CHECK** ck_file_versions_size CHECK (((size_bytes >= 1) AND (size_bytes <= 26214400)))

**CHECK** ck_file_versions_version CHECK (version >= 1)

**collab.file_comments**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| file_id | uuid | | FK | | |
| file_version_id | uuid | ✓ | FK | | |
| author_person_type | varchar(10) | | | | |
| author_person_id | uuid | | | | |
| author_name | varchar(150) | | | | |
| body | varchar(2000) | | | | |
| created_at | timestamptz | | | now() | |

**PK** pk_file_comments PRIMARY KEY (id)

**FK** fk_file_comments_file FOREIGN KEY (file_id) REFERENCES collab.project_files(id) ON DELETE CASCADE

**FK** fk_file_comments_version FOREIGN KEY (file_version_id) REFERENCES collab.file_versions(id) ON DELETE SET NULL

**CHECK** ck_file_comments_author_type CHECK ((author_person_type IN ('STUDENT', 'LECTURER', 'MENTOR')))

**INDEX** idx_file_comments_file btree (file_id, created_at)

**collab.message_attachments**

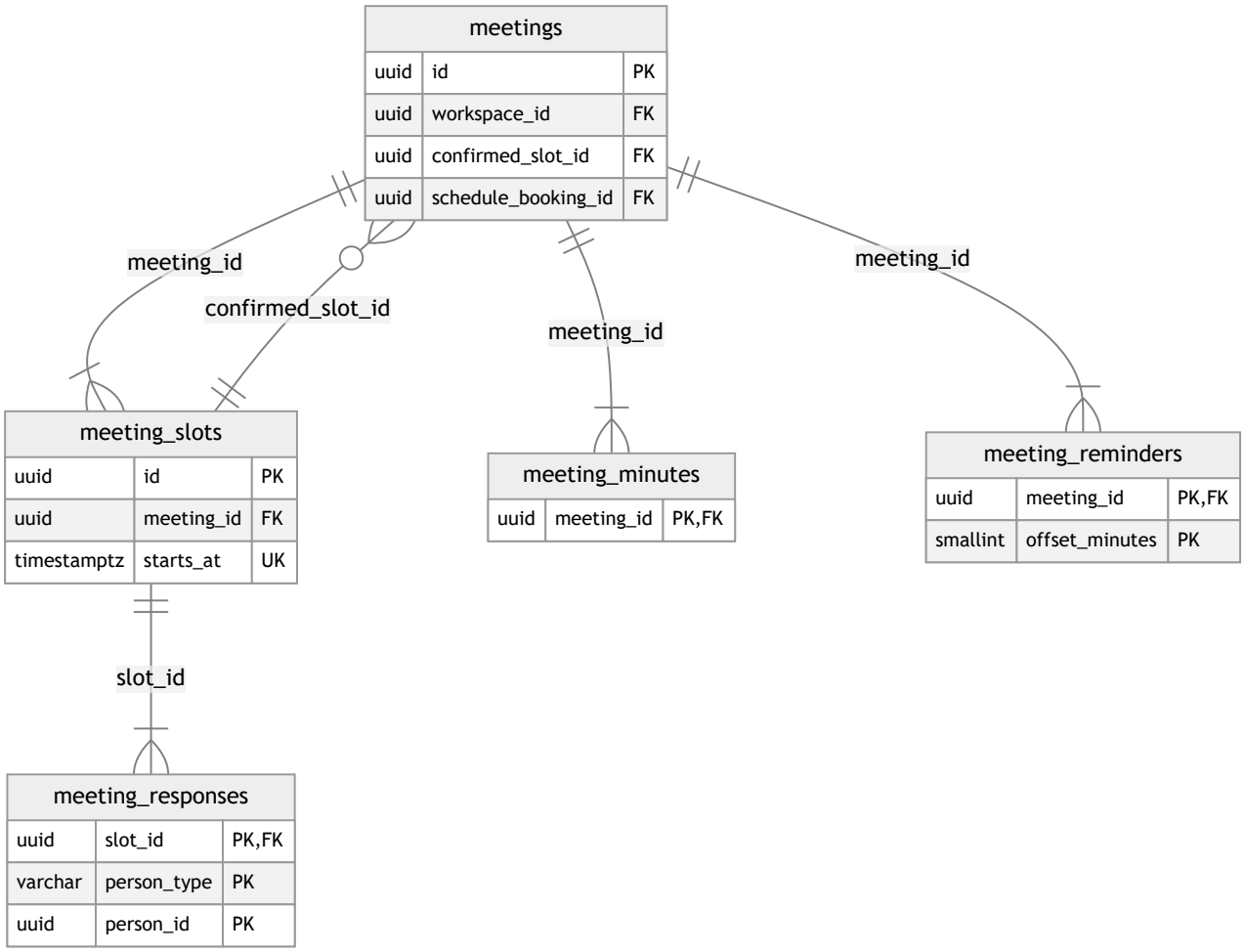
| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| message_id | uuid | | PK FK | | |
| file_id | uuid | | PK FK | | |

**PK** pk_message_attachments PRIMARY KEY (message_id, file_id)

**FK** fk_message_attachments_file FOREIGN KEY (file_id) REFERENCES collab.project_files(id) ON DELETE CASCADE

**FK** fk_message_attachments_message FOREIGN KEY (message_id) REFERENCES collab.messages(id) ON DELETE CASCADE

## Lịch họp (rule 25)




**collab.meetings**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| workspace_id | uuid | | FK | | |
| title | varchar(200) | | | | |
| agenda | varchar(2000) | ✓ | | | |
| location | varchar(500) | ✓ | | | phòng hoặc link họp nhập tay |
| organizer_type | varchar(10) | | | | |
| organizer_id | uuid | | | | |
| organizer_name | varchar(150) | | | | |
| organizer_role | varchar(10) | | | | |
| is_mentor_session | boolean | | | false | họp do mentor tổ chức = buổi mentor |
| status | varchar(10) | | | PROPOSED | |
| confirmed_slot_id | uuid | ✓ | FK | | |
| cancel_reason | varchar(500) | ✓ | | | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| schedule_booking_id | uuid | ✓ | FK | | |
| week_no | smallint | ✓ | | | |

**PK** pk_meetings PRIMARY KEY (id)

**FK** fk_meetings_confirmed_slot FOREIGN KEY (confirmed_slot_id) REFERENCES collab.meeting_slots(id) ON DELETE RESTRICT

**FK** fk_meetings_schedule_booking FOREIGN KEY (schedule_booking_id) REFERENCES collab.slot_bookings(id) ON DELETE SET NULL

**FK** fk_meetings_workspace FOREIGN KEY (workspace_id) REFERENCES collab.workspaces(id) ON DELETE CASCADE

**CHECK** ck_meetings_cancel CHECK ((((status) = 'CANCELLED') = (cancel_reason IS NOT NULL)))

**CHECK** ck_meetings_confirmed CHECK (((status IN ('SCHEDULED', 'DONE')) = (confirmed_slot_id IS NOT NULL)))

**CHECK** ck_meetings_mentor_session CHECK ((is_mentor_session = ((organizer_role) = 'MENTOR')))

**CHECK** ck_meetings_organizer_role CHECK ((organizer_role IN ('LEADER', 'INSTRUCTOR', 'MENTOR')))

**CHECK** ck_meetings_status CHECK ((status IN ('PROPOSED', 'SCHEDULED', 'CANCELLED', 'DONE')))

**CHECK** ck_meetings_weekly CHECK ((((schedule_booking_id IS NULL) AND (week_no IS NULL)) OR (((week_no >= 1) AND (week_no <= 20)) AND ((organizer_role) = 'INSTRUCTOR'))))

**INDEX** idx_meetings_workspace_status btree (workspace_id, status)

**UNIQUE INDEX** uq_meetings_weekly btree (schedule_booking_id, week_no) WHERE (schedule_booking_id IS NOT NULL)

**TRIGGER** trg_meetings_updated_at BEFORE UPDATE ON collab.meetings FOR EACH ROW EXECUTE FUNCTION collab.set_updated_at()

**collab.meeting_slots**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| meeting_id | uuid | | UQ FK | | |
| starts_at | timestamptz | | UQ | | |
| duration_minutes | smallint | | | 60 | |

**UNIQUE** uq_meeting_slots_meeting_start UNIQUE (meeting_id, starts_at)

**PK** pk_meeting_slots PRIMARY KEY (id)

**FK** fk_meeting_slots_meeting FOREIGN KEY (meeting_id) REFERENCES collab.meetings(id) ON DELETE CASCADE

**CHECK** ck_meeting_slots_duration CHECK (((duration_minutes >= 15) AND (duration_minutes <= 240)))

**INDEX** idx_meeting_slots_starts btree (starts_at)

**collab.meeting_responses**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| slot_id | uuid | | PK FK | | |
| person_type | varchar(10) | | PK | | |
| person_id | uuid | | PK | | |
| answer | varchar(3) | | | | |
| answered_at | timestamptz | | | now() | |

**PK** pk_meeting_responses PRIMARY KEY (slot_id, person_type, person_id)

**FK** fk_meeting_responses_slot FOREIGN KEY (slot_id) REFERENCES collab.meeting_slots(id) ON DELETE CASCADE

**CHECK** ck_meeting_responses_answer CHECK ((answer IN ('YES', 'NO')))

**collab.meeting_minutes**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| meeting_id | uuid | | PK FK | | |
| content | varchar(8000) | | | | |
| recorded_by_id | uuid | | | | |
| recorded_by_name | varchar(150) | | | | |
| recorded_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**PK** pk_meeting_minutes PRIMARY KEY (meeting_id)

**FK** fk_meeting_minutes_meeting FOREIGN KEY (meeting_id) REFERENCES collab.meetings(id) ON DELETE CASCADE

**TRIGGER** trg_meeting_minutes_updated_at BEFORE UPDATE ON collab.meeting_minutes FOR EACH ROW EXECUTE FUNCTION collab.set_updated_at()

**collab.meeting_reminders**

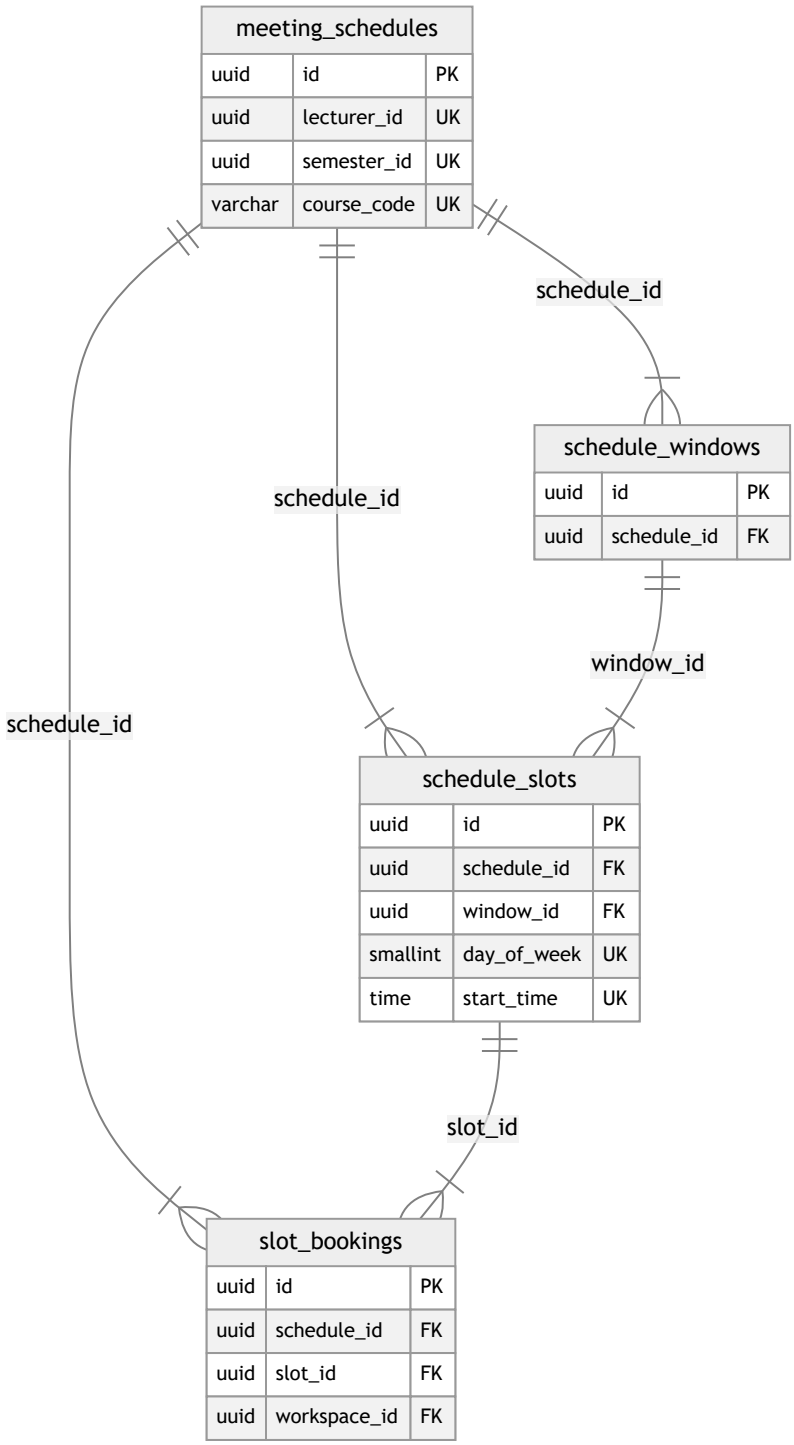
Job Service gọi /internal/meetings/upcoming; mỗi mốc nhắc (1440, 60 phút) chỉ gửi một lần.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| meeting_id | uuid | | PK FK | | |
| offset_minutes | smallint | | PK | | |
| sent_at | timestamptz | | | now() | |

**PK** pk_meeting_reminders PRIMARY KEY (meeting_id, offset_minutes)

**FK** fk_meeting_reminders_meeting FOREIGN KEY (meeting_id) REFERENCES collab.meetings(id) ON DELETE CASCADE

## Lịch họp hằng tuần EXE201 (rule 25)




**`collab.meeting_schedules`**

Giảng viên hướng dẫn đăng các khung giờ, hệ thống cắt thành slot (mặc định 15 phút); mỗi nhóm đặt một slot cố định cho cả kỳ, ai đặt trước được trước. Từ slot đã đặt, Collaboration sinh một cuộc họp mỗi tuần (meetings.schedule_booking_id).

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| `id` | `uuid` | | PK | `gen_random_uuid()` | |
| `lecturer_id` | `uuid` | | UQ | | academic.lecturers.id |
| `lecturer_name` | `varchar(150)` | | | | snapshot |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| semester_id | uuid | | UQ | | |
| course_code | varchar(6) | | UQ | EXE201 | |
| slot_minutes | smallint | | | 15 | |
| location | varchar(500) | | | | link họp online |
| booking_deadline | timestamptz | | | | |
| status | varchar(10) | | | DRAFT | |
| published_at | timestamptz | ✓ | | | |
| closed_at | timestamptz | ✓ | | | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**UNIQUE** uq_meeting_schedules_lecturer UNIQUE (lecturer_id, semester_id, course_code)

**PK** pk_meeting_schedules PRIMARY KEY (id)

**CHECK** ck_meeting_schedules_closed CHECK ((((status) = 'CLOSED') = (closed_at IS NOT NULL)))

**CHECK** ck_meeting_schedules_course CHECK (((course_code) = 'EXE201'))

**CHECK** ck_meeting_schedules_published CHECK ((((status) = 'DRAFT') = (published_at IS NULL)))

**CHECK** ck_meeting_schedules_slot CHECK (((slot_minutes >= 5) AND (slot_minutes <= 60)))

**CHECK** ck_meeting_schedules_status CHECK ((status IN ('DRAFT', 'PUBLISHED', 'CLOSED')))

**TRIGGER** trg_meeting_schedules_updated_at BEFORE UPDATE ON collab.meeting_schedules FOR EACH ROW EXECUTE FUNCTION collab.set_updated_at()

#### collab.schedule_windows

Khung giờ trong tuần, ví dụ thứ Ba 09:00–11:00 (day_of_week: 1 = thứ Hai … 7 = Chủ nhật, giờ địa phương).

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| schedule_id | uuid | | FK | | |
| day_of_week | smallint | | | | |
| start_time | time | | | | |
| end_time | time | | | | |

**PK** pk_schedule_windows PRIMARY KEY (id)

**FK** fk_schedule_windows_schedule FOREIGN KEY (schedule_id) REFERENCES collab.meeting_schedules(id) ON DELETE CASCADE

**CHECK** ck_schedule_windows_day CHECK (((day_of_week >= 1) AND (day_of_week <= 7)))

**CHECK** ck_schedule_windows_time CHECK (end_time > start_time)

**INDEX** idx_schedule_windows_schedule btree (schedule_id)

#### collab.schedule_slots

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| schedule_id | uuid | | UQ FK | | |
| window_id | uuid | | FK | | |
| day_of_week | smallint | | UQ | | |
| start_time | time | | UQ | | |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| end_time | time | | | | |

**UNIQUE** uq_schedule_slots_time UNIQUE (schedule_id, day_of_week, start_time)

**PK** pk_schedule_slots PRIMARY KEY (id)

**FK** fk_schedule_slots_schedule FOREIGN KEY (schedule_id) REFERENCES collab.meeting_schedules(id) ON DELETE CASCADE

**FK** fk_schedule_slots_window FOREIGN KEY (window_id) REFERENCES collab.schedule_windows(id) ON DELETE CASCADE

**CHECK** ck_schedule_slots_day CHECK (((day_of_week >= 1) AND (day_of_week <= 7)))

**CHECK** ck_schedule_slots_time CHECK (end_time > start_time)

#### collab.slot_bookings

Một slot một nhóm, một nhóm một slot đang hiệu lực. Đổi slot = hủy dòng cũ (cancelled_at) rồi tạo dòng mới.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| schedule_id | uuid | | FK | | |
| slot_id | uuid | | FK | | |
| workspace_id | uuid | | FK | | |
| source | varchar(10) | | | | LEADER tự đặt, INSTRUCTOR đổi giúp, AUTO xếp lúc hết hạn |
| booked_by_id | uuid | ✓ | | | sinh viên / giảng viên; NULL khi AUTO |
| booked_at | timestamptz | | | now() | |
| cancelled_at | timestamptz | ✓ | | | |

**PK** pk_slot_bookings PRIMARY KEY (id)

**FK** fk_slot_bookings_schedule FOREIGN KEY (schedule_id) REFERENCES collab.meeting_schedules(id) ON DELETE CASCADE

**FK** fk_slot_bookings_slot FOREIGN KEY (slot_id) REFERENCES collab.schedule_slots(id) ON DELETE CASCADE

**FK** fk_slot_bookings_workspace FOREIGN KEY (workspace_id) REFERENCES collab.workspaces(id) ON DELETE CASCADE

**CHECK** ck_slot_bookings_by CHECK ((((source) = 'AUTO') = (booked_by_id IS NULL)))

**CHECK** ck_slot_bookings_source CHECK ((source IN ('LEADER', 'INSTRUCTOR', 'AUTO')))

**UNIQUE INDEX** uq_slot_bookings_slot_active btree (slot_id) WHERE (cancelled_at IS NULL)

**UNIQUE INDEX** uq_slot_bookings_workspace_active btree (schedule_id, workspace_id) WHERE (cancelled_at IS NULL)

## Checkpoint EXE101 (rule 27)

#### collab.checkpoint_reviews

Giảng viên đánh dấu nhóm đã trình bày Checkpoint 1–4 trên lớp. Không lưu điểm (điểm ở FAP). Bỏ đánh dấu (trong ngày) = xóa dòng; Services kiểm tra giới hạn trong ngày.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| workspace_id | uuid | | UQ FK | | |
| checkpoint_no | smallint | | UQ | | |
| presented_on | date | | | | |
| comment | varchar(500) | ✓ | | | |
| reviewed_by_lecturer_id | uuid | | | | |
| reviewed_by_name | varchar(150) | | | | snapshot |
| created_at | timestamptz | | | now() | |

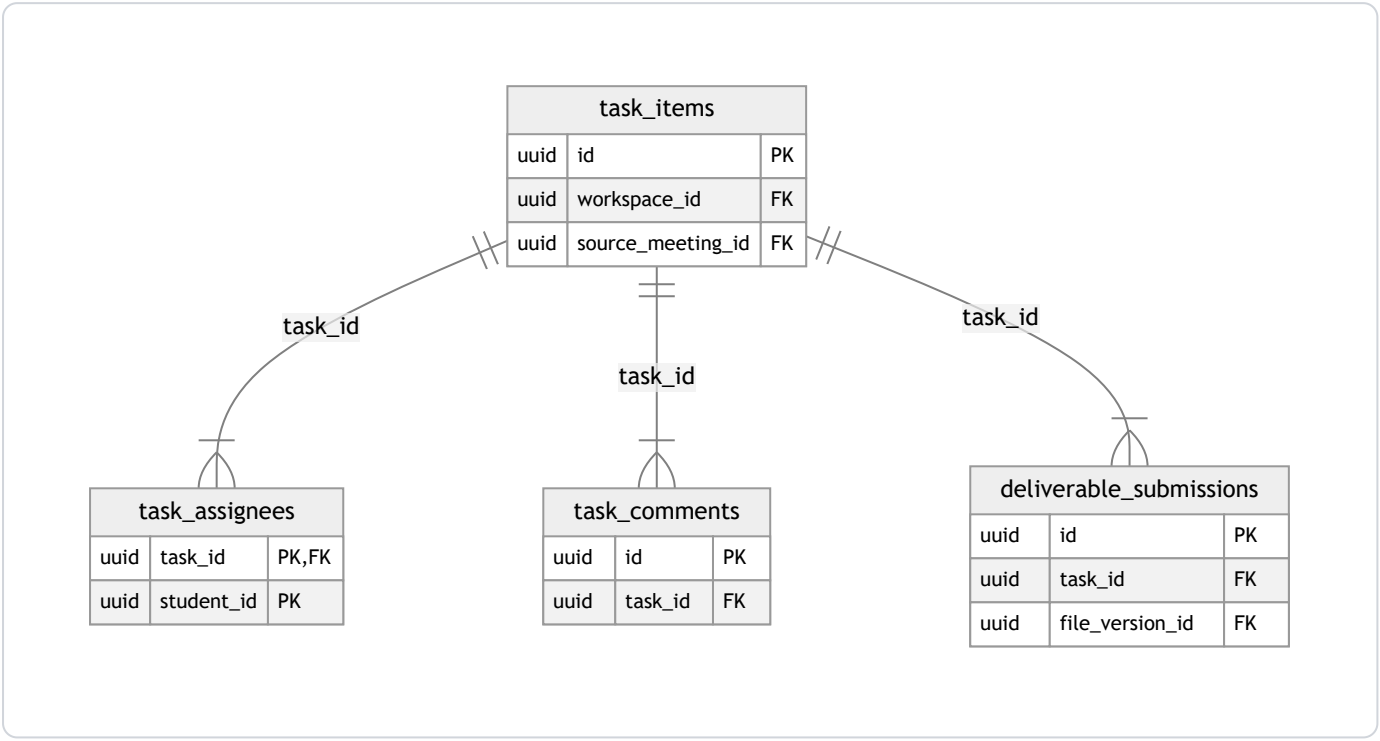
**UNIQUE** uq_checkpoint_reviews_workspace UNIQUE (workspace_id, checkpoint_no)

**PK** pk_checkpoint_reviews PRIMARY KEY (id)

**FK** fk_checkpoint_reviews_workspace FOREIGN KEY (workspace_id) REFERENCES collab.workspaces(id) ON DELETE CASCADE

**CHECK** ck_checkpoint_reviews_no CHECK (((checkpoint_no >= 1) AND (checkpoint_no <= 4)))

## Việc và bài nộp (rule 23)




**collab.task_items**

kind = DELIVERABLE: bài nộp do giảng viên hướng dẫn tạo, cần nộp file.

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| workspace_id | uuid | | FK | | |
| kind | varchar(12) | | | TASK | |
| title | varchar(200) | | | | |
| description | varchar(4000) | ✓ | | | |
| status | varchar(5) | | | TODO | |
| due_at | timestamptz | ✓ | | | |
| position | integer | | | 0 | thứ tự trong cột |
| created_by_type | varchar(10) | | | | |
| created_by_id | uuid | | | | |
| source_meeting_id | uuid | ✓ | FK | | tạo từ biên bản họp |
| completed_at | timestamptz | ✓ | | | |
| created_at | timestamptz | | | now() | |
| updated_at | timestamptz | | | now() | |

**PK** pk_task_items PRIMARY KEY (id)

**FK** fk_task_items_meeting FOREIGN KEY (source_meeting_id) REFERENCES collab.meetings(id) ON DELETE SET NULL

**FK** fk_task_items_workspace FOREIGN KEY (workspace_id) REFERENCES collab.workspaces(id) ON DELETE CASCADE

**CHECK** ck_task_items_creator CHECK ((created_by_type IN ('STUDENT', 'LECTURER', 'MENTOR')))

**CHECK** ck_task_items_deliverable_creator CHECK ((((kind) <> 'DELIVERABLE') OR ((created_by_type) = 'LECTURER')))

**CHECK** ck_task_items_deliverable_due CHECK ((((kind) <> 'DELIVERABLE') OR (due_at IS NOT NULL)))

**CHECK** ck_task_items_done CHECK ((((status) = 'DONE') = (completed_at IS NOT NULL)))

**CHECK** ck_task_items_kind CHECK ((kind IN ('TASK', 'DELIVERABLE')))

**CHECK** ck_task_items_status CHECK ((status IN ('TODO', 'DOING', 'DONE')))

**CHECK** ck_task_items_title CHECK ((char_length(btrim(title)) >= 1))

**INDEX** idx_task_items_board btree (workspace_id, status, "position")

**INDEX** idx_task_items_due btree (due_at) WHERE (((status) <> 'DONE') AND (due_at IS NOT NULL))

**TRIGGER** trg_task_items_updated_at BEFORE UPDATE ON collab.task_items FOR EACH ROW EXECUTE FUNCTION collab.set_updated_at()

**collab.task_assignees**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| task_id | uuid | | PK FK | | |
| student_id | uuid | | PK | | |

**PK** pk_task_assignees PRIMARY KEY (task_id, student_id)

**FK** fk_task_assignees_task FOREIGN KEY (task_id) REFERENCES collab.task_items(id) ON DELETE CASCADE

**INDEX** idx_task_assignees_student btree (student_id)

**collab.task_comments**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| task_id | uuid | | FK | | |
| author_person_type | varchar(10) | | | | |
| author_person_id | uuid | | | | |
| author_name | varchar(150) | | | | |
| body | varchar(2000) | | | | |
| created_at | timestamptz | | | now() | |

**PK** pk_task_comments PRIMARY KEY (id)

**FK** fk_task_comments_task FOREIGN KEY (task_id) REFERENCES collab.task_items(id) ON DELETE CASCADE

**CHECK** ck_task_comments_author_type CHECK ((author_person_type IN ('STUDENT', 'LECTURER', 'MENTOR')))

**INDEX** idx_task_comments_task btree (task_id, created_at)

**collab.deliverable_submissions**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | gen_random_uuid() | |
| task_id | uuid | | FK | | |
| file_version_id | uuid | | FK | | |
| submitted_by_id | uuid | | | | students.id |
| submitted_at | timestamptz | | | now() | |
| review_status | varchar(15) | | | PENDING | |
| review_comment | varchar(2000) | ✓ | | | |
| reviewed_by_id | uuid | ✓ | | | lecturers.id |

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| reviewed_at | timestamptz | ✓ | | | |

**PK** pk_deliverable_submissions PRIMARY KEY (id)

**FK** fk_deliverable_submissions_task FOREIGN KEY (task_id) REFERENCES collab.task_items(id) ON DELETE CASCADE

**FK** fk_deliverable_submissions_version FOREIGN KEY (file_version_id) REFERENCES collab.file_versions(id) ON DELETE RESTRICT

**CHECK** ck_deliverable_submissions_review CHECK ((((review_status) = 'PENDING') = ((reviewed_at IS NULL) AND (reviewed_by_id IS NULL))))

**CHECK** ck_deliverable_submissions_revision_note CHECK ((((review_status) <> 'NEEDS_REVISION') OR (review_comment IS NOT NULL)))

**CHECK** ck_deliverable_submissions_status CHECK ((review_status IN ('PENDING', 'ACCEPTED', 'NEEDS_REVISION')))

**INDEX** idx_deliverable_submissions_pending btree (task_id) WHERE ((review_status) = 'PENDING')

**INDEX** idx_deliverable_submissions_task btree (task_id, submitted_at DESC)

**TRIGGER** trg_deliverable_submissions_kind BEFORE INSERT OR UPDATE OF task_id ON collab.deliverable_submissions FOR EACH ROW EXECUTE FUNCTION collab.tg_submission_is_deliverable()

## Outbox (topic exe.collab), idempotency khi consume

**collab.outbox_messages**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| id | uuid | | PK | | = eventId của envelope |
| topic | varchar(100) | | | | |
| event_type | varchar(100) | | | | |
| aggregate_type | varchar(50) | | | | |
| aggregate_id | uuid | | | | Kafka message key |
| payload | jsonb | | | | |
| correlation_id | varchar(64) | ✓ | | | |
| occurred_at | timestamptz | | | now() | |
| processed_at | timestamptz | ✓ | | | |
| attempts | integer | | | 0 | |
| last_error | text | ✓ | | | |

**PK** pk_outbox_messages PRIMARY KEY (id)

**CHECK** ck_outbox_messages_attempts CHECK (attempts >= 0)

**INDEX** idx_outbox_messages_pending btree (occurred_at) WHERE (processed_at IS NULL)

**collab.processed_events**

| Cột | Kiểu | Null | Khóa | Mặc định | Ghi chú |
|---|---|---|---|---|---|
| event_id | uuid | | PK | | |
| event_type | varchar(100) | | | | |
| processed_at | timestamptz | | | now() | |

**PK** pk_processed_events PRIMARY KEY (event_id)

# 9. Service không có schema nghiệp vụ

## 9.1 Matching Service — Redis read model

Matching chỉ tính toán (gRPC), không có SQL. Read model trong Redis cập nhật từ Kafka; dựng lại được bằng cách replay `exe.student`, `exe.team`, `exe.assignment` từ đầu. Mỗi event lưu `matching:processed:{eventId}` (TTL 7 ngày) để idempotent.

| Key | Kiểu | Nội dung | Cập nhật bởi |
|---|---|---|---|
| `matching:{semesterId}:student:{studentId}` | Hash | chuyên ngành, 5 kỹ năng, teamId (rỗng = chưa có nhóm) | StudentImported, StudentProfileUpdated, MemberJoined, MemberLeft |
| `matching:{semesterId}:unmatched:{specializationId}` | Set | sinh viên chưa có nhóm theo chuyên ngành | StudentImported, MemberJoined, MemberLeft |
| `matching:{semesterId}:team:{teamId}` | Hash | số thành viên, chuyên ngành, kỹ năng (max mỗi loại), domain, trạng thái, giảng viên / mentor đã ghép | TeamCreated, MemberJoined, MemberLeft, TeamFinalized, ProjectChanged, LecturerAssigned, MentorAssigned |
| `matching:advisor:{kind}:{id}` | Hash | domain chuyên môn, số nhóm tối đa / mong muốn, tải hiện tại | LecturerRegistered, LecturerProfileUpdated, MentorRegistered, LecturerAssigned, MentorAssigned, MentorUnassigned |
| `matching:{semesterId}:interest:{teamId}` | Set | cố vấn đang quan tâm nhóm | TopicInterestOpened, TopicInterestClosed |
| `matching:{semesterId}:prefs:{teamId}` | List | nguyện vọng giảng viên 1–3 | LecturerPreferencesUpdated |

## 9.2 Job Service — schema hangfire

`UsePostgreSqlStorage(..., new PostgreSqlStorageOptions { SchemaName = "hangfire", PrepareSchemaIfNecessary = true })` với role `job_svc`; script `05_job_db.sql` chỉ tạo schema. Hangfire tự tạo bảng; không có bảng nghiệp vụ. Job gọi endpoint nội bộ của service sở hữu dữ liệu (service token) thay vì ghi DB.

# 10. Tham chiếu chéo service và đồng bộ qua Kafka

## 10.1 ID tham chiếu sang service khác

| Cột | Trỏ tới (chủ sở hữu) | Kiểm tra khi ghi | Đồng bộ |
|---|---|---|---|
| `team.*.student_id, invitee_student_id, requested_by_student_id` | `academic.students.id` | Team gọi Academic HTTP "SV có trong học kỳ?", lấy snapshot MSSV, tên, chuyên ngành | Snapshot lúc ghi; khi chốt nhóm lấy lại chuyên ngành |
| `team.teams.semester_id, *.round_id` | `academic.semesters / registration_rounds` | HTTP "đợt có đang mở?" | Không cần |
| `team.*.specialization_id, domain_id (+ code)` | `academic.specializations / domains` | HTTP lấy danh mục (Redis cache) | Danh mục chỉ vô hiệu, không xóa |
| `team.lecturer_preferences / lecturer_assignments.lecturer_id, topic_interests.advisor_id, mentor_assignments.mentor_id` | `academic.lecturers / mentors` | HTTP kiểm tra cố vấn đang hoạt động, số nhóm tối đa | Snapshot tên cố vấn |
| `collab.workspaces.team_id, project_id; workspace_members.person_id` | `team.teams, team.projects; academic students / lecturers / mentors` | — | Consume exe.team, exe.assignment |
| `*_account_id, created_by, imported_by, updated_by` | `identity.user_accounts.id` | Lấy từ claim sub trong JWT | Không cần |
| `identity.user_account_roles.external_ref_id` | `academic students / lecturers / mentors` | — | Consume StudentImported, LecturerRegistered, MentorRegistered |
| `notification.contacts.id, notifications.recipient_id` | `academic students / lecturers / mentors` | — | Consume exe.student |

JWT do Identity cấp mang `sub` (user_account id), `roles` (danh sách) và `student_id / lecturer_id / mentor_id` (= external_ref_id), nên Team, Collaboration và Notification làm việc trực tiếp bằng ID của Academic.

## 10.2 Topic, producer và consumer

| Topic | Event | Producer | Consumer |
|---|---|---|---|
| `exe.student` | StudentImported, StudentProfileUpdated, LecturerRegistered, LecturerProfileUpdated, MentorRegistered | Academic (outbox) | Identity, Notification, Matching |
| `exe.registration` | RoundOpened, DeadlineApproaching, RoundClosed | Academic (outbox), Job | Notification |
| `exe.team` | TeamCreated, MemberJoined, MemberLeft, LeaderChanged, TeamFinalized, TeamNeedsReview, ProjectCreated, ProjectChanged, TeamLinkedToProject | Team (outbox) | Notification, Matching, Collaboration |
| `exe.invitation` | InvitationSent / Accepted / Declined / Expired, JoinRequestSubmitted / Accepted / Declined | Team (outbox) | Notification |
| `exe.change-request` | ChangeRequestSubmitted / Approved / Rejected | Team (outbox) | Notification |
| `exe.assignment` | TopicInterestOpened / Closed, LecturerPreferencesUpdated, LecturerAssigned, MentorAssigned, MentorUnassigned, AdvisorMatchingCompleted | Team (outbox) | Notification, Matching, Collaboration |
| `exe.collab` | AdvisorQuestionAsked / Answered, UserMentioned, TaskAssigned, Deliverable*, FileUploaded, Meeting*, MeetingSlotsPublished, MeetingSlotBooked, CheckpointReviewed | Collaboration (outbox) | Notification |
| `exe.reminder` | MeetingReminder, TaskDueSoon, DeliverableOverdue | Job (publish trực tiếp) | Notification |

### 10.3 Transactional Outbox và consumer idempotent

- Academic, Team, Collaboration có `outbox_messages`: thay đổi nghiệp vụ + audit log + dòng outbox ghi trong một transaction. `id` là eventId của envelope; `aggregate_id` làm Kafka key để giữ thứ tự theo đối tượng.
- Dispatcher nền mỗi 1–2 giây: `... WHERE processed_at IS NULL ORDER BY occurred_at LIMIT 100 FOR UPDATE SKIP LOCKED` → publish → `processed_at`; lỗi thì tăng `attempts`, ghi `last_error`.
- Identity, Notification, Collaboration lưu eventId trong `processed_events` cùng transaction với thay đổi; Notification còn có `uq_notifications_event_recipient`, `uq_email_logs_event_to`. Lỗi sau khi retry → `.dlq`.
- Job không có outbox: event của nó không gắn với thay đổi dữ liệu nào, publish trực tiếp là đủ.

## 11. Thay đổi so với bản 19/10/2026

Tóm tắt; chi tiết từng dòng ở `deploy/db/README.md` mục "Khác với PDF thiết kế".

| Chỗ | Thay đổi | Quy tắc |
|---|---|---|
| Database mới | `collab_db` (role `collab_svc`): workspace, tin nhắn, file, việc, họp, lịch tuần, checkpoint | §5.8, 21–25, 27 |
| Vai trò tài khoản | `user_accounts.role_id` → bảng `user_account_roles`; thêm role Mentor | 5, 20 |
| Hồ sơ sinh viên | `student_skills` (5 kỹ năng 1–5), `description` ≤ 500, `profile_completed_at`, `is_seeking_team` | 16 |
| Giảng viên, mentor | `lecturer_domains`, `max_teams_exe101 / _exe201`; `mentors` (`max_teams`, `preferred_teams`, `is_internal`), `mentor_domains` | 9, 19, 20 |
| Học kỳ, cấu hình | `mentoring_start_week`, `expected_mentor_sessions`, `slot_booking_deadline_week`, `meeting_slot_minutes`, `checkpoint_target_weeks`; bảng `rule_settings` | 9, 17, 19, 20, 24, 25, 27 |
| Trạng thái nhóm | FORMING / FINALIZED / DISBANDED → DRAFT / ACTIVE / NEEDS_REVIEW / FINALIZED / DISBANDED | 13–15 |
| Đề tài xuyên kỳ | `projects`, `project_versions`, `teams.project_id` (thay `idea_title`); item PROJECT_CHANGE | 26 |
| Mốc EXE101, đổi nhóm | `team_baseline_members`, `teams.is_inherited`; item chờ không cần change request; `round_id`, số và % thay đổi | 7 |
| Ý tưởng | `join_requests.has_no_idea`, `join_request_ideas` | 17 |
| Ghép cố vấn | `topic_interests`; `lecturer_preferences` không trạng thái; `lecturer_assignments` ASSIGNED / REVOKED + reason, source; `mentor_assignments`; `advisor_matching_runs` | 9, 19, 20 |
| Thông báo | `contacts.email_enabled`; người nhận có thể là mentor | A07 |